API REFERENCE — Modulo Booking

Backend Gestionale B&B Cosenza Versione API `v1` · Documento aggiornato al **10 ottobre 2026** · Copertura: Step A → G

> I pagamenti online hanno un documento proprio: `payment_api.md`.

Come usare questo documento

Serve a tre scopi, e ogni sezione è pensata per uno di essi.

Per il collaudo manuale: la sezione *Endpoint* descrive ogni rotta con i suoi codici di risposta e un esempio `curl` pronto da incollare. La *Checklist di verifica* in fondo è la sequenza da seguire a sviluppo concluso.

Per lo sviluppo frontend: la sezione *Schemi* elenca ogni campo con tipo, vincoli e obbligatorietà. La sezione *Meccanismi trasversali* spiega cosa il client deve inviare oltre ai dati veri e propri (token del preventivo, captcha, honeypot) e come interpretare gli errori.

Per chi riprende lo sviluppo: la *Logica* di ciascun endpoint dice cosa accade dietro la chiamata e quali test la coprono.

> Questo documento viene aggiornato a ogni modifica del modulo. Il *Changelog* in fondo traccia cosa è cambiato e quando.

---

1. Convenzioni generali

1.1 Base URL e versioning

```
http://localhost:8000/api/v1
```

Tutte le rotte del modulo sono sotto `/api/v1/bookings`.

1.2 Formato delle date

Campo	Formato	Esempio
Date di soggiorno (`check_in`, `check_out`)	`YYYY-MM-DD` (solo data)	`2026-12-01`
Istanti (`hold_expires_at`, `confirmed_at`, …)	ISO 8601 con fuso, in UTC	`2026-12-01T14:30:00Z`

> ⚠️ **Nota per il frontend.** Le date di soggiorno **non** sono timestamp. `check_out` è il giorno di partenza ed è **escluso** dal soggiorno: `2026-12-01 → 2026-12-03` sono **due notti**. Prima dello Step A questi campi erano timestamp completi: se esiste codice Angular che li tratta come tali, va adeguato.

1.3 Importi

Tutti gli importi sono **stringhe decimali con due cifre** (`"200.00"`), non numeri in virgola mobile. Serializzarli come `float` in JavaScript introduce errori di arrotondamento sui totali.

La valuta è in `currency` (attualmente sempre `"EUR"`).

1.4 Formato degli errori

Ogni errore ha la stessa forma:

```
{
  "message": "Testo leggibile, già in italiano, mostrabile all'utente"
}
```

Gli errori di validazione hanno un campo aggiuntivo:

```
{
  "message": "La password deve contenere almeno una maiuscola",
  "details": [
    { "type": "value_error", "loc": ["body", "password"], "msg": "..." }
  ]
}
```

`details` serve al frontend per evidenziare il campo in errore (`loc` contiene il percorso). **Il valore rifiutato non viene mai restituito**: `details` non contiene la chiave `input` , per evitare che una password o un dato personale torni indietro nella risposta.

1.5 Codici di stato e loro significato

Codice	Quando	Cosa deve fare il client
`200`	Operazione riuscita	—
`201`	Prenotazione creata	—
`400`	Token di conferma o cancellazione non valido	Invitare a richiedere un nuovo link
`401`	Sessione assente o scaduta	Reindirizzare al login
`403`	Ruolo insufficiente	Nascondere la funzione
`404`	Risorsa inesistente, o non di proprietà dell'utente	Messaggio generico
`409`	Conflitto: slot occupato, transizione non ammessa, già confermata	Mostrare il messaggio e proporre una nuova ricerca
`410`	Blocco temporaneo scaduto	Invitare a ripetere la prenotazione
`422`	Dati non validi, preventivo scaduto, captcha fallito	Evidenziare il campo tramite `details`
`429`	Troppe richieste	Attendere i secondi indicati in `Retry-After`
`402`	Pagamento richiesto o fallito *(dallo Step G)*	—
`500`	Errore interno, **compreso un modello di risposta costruito male dal server**	Riprovare; il dettaglio è nei log del backend, non nella risposta

> **Un `422` è sempre colpa del client.** Non era vero fino al 26/09/2026: la stessa `ValidationError` di Pydantic nasce sia quando il client manda dati sbagliati, sia quando il *server* compone male una risposta, ed entrambe uscivano come `422` *"il campo X è obbligatorio"*. Chi riceveva quel messaggio cercava nel proprio payload un campo che non aveva mai inviato. Ora il confine è tracciato dal tipo dell'eccezione: `RequestValidationError` → `422`, qualunque altra `ValidationError` → `500` con il dettaglio nei log. Se il frontend riceve un `422`, il campo indicato è davvero nella richiesta.

1.6 Mappa eccezione → codice

Il backend non espone mai stack trace. Ogni eccezione di dominio deriva da `AppException` e porta con sé il proprio codice HTTP; un **unico handler** globale le traduce in risposta.

Eccezione	Codice	Messaggio predefinito
`EntityNotFound`	404	Entity not found
`EntityAlreadyExists`	409	Entity already exists
`EntityInUse`	409	L'elemento è utilizzato da altri dati e non può essere eliminato
`RoomNotAvailable`	409	Una o più camere non sono disponibili per le date richieste
`InvalidBookingStatusTransition`	409	L'operazione non è consentita nello stato attuale
`BookingNotCancellable`	409	La prenotazione non può più essere cancellata
`StatusChangeReasonRequired`	422	Per questa operazione serve una motivazione *(es. partenza anticipata)*
`BookingHoldExpired`	410	Il tempo per completare la prenotazione è scaduto
`InvalidBookingToken`	400	Il link utilizzato non è valido o è già stato usato
`InvalidDateRange`	422	L'intervallo di date indicato non è valido
`InvalidGuestCount`	422	Il numero di ospiti non è compatibile con le camere
`InvalidQuoteToken`	422	Il preventivo non è più valido
`CaptchaValidationFailed`	422	Verifica anti-bot non superata
`ConcurrentModification`	409	La prenotazione è stata modificata da un altro operatore
`InvalidPaymentOperation`	409	L'operazione non è compatibile con lo stato del pagamento
`RateLimitExceeded`	429	Troppe richieste: riprova più tardi
`InvalidFileType` / `InvalidFileSize`	422	—
`StaleDataError` *(SQLAlchemy)*	409	Il dato è stato modificato da un'altra operazione
`IntegrityError` *(SQLAlchemy)* — sovrapposizione di notti (`23P01`)	409	Una o più camere non sono disponibili per le date richieste *(dal 09/10/2026)*
`IntegrityError` — valore duplicato (`23505`) o riferimento mancante (`23503`)	409	L'operazione è in conflitto con dati già presenti. Ricarica la pagina e riprova. *(dal 09/10/2026)*
`IntegrityError` — ogni altro vincolo	**500**	Errore interno del server *(dal 09/10/2026)*
`RequestValidationError` *(FastAPI)*	422	Dipende dal campo, con `details`
`ValidationError` *(Pydantic, altrove)* · `ResponseValidationError`	**500**	Errore interno del server — dettaglio solo nei log

2. Autenticazione e autorizzazione

2.1 Come funziona

Il progetto usa **JWT veicolati da cookie** `HttpOnly`, non header `Authorization`. Il token non è leggibile da JavaScript, il che elimina il furto per Cross-Site Scripting.

Login su `POST /api/v1/auth/token` con form data (`username` = email, `password`). La risposta imposta due cookie:

Cookie	Durata	Attributi
`access_token`	breve (`access_token_expire_minutes`)	`HttpOnly`, `SameSite=lax`, `Path=/`
`refresh_token`	lunga (`refresh_token_expire_minutes`)	`HttpOnly`, `SameSite=lax`, `Path=/`

> ⚠️ **Frontend**: tutte le chiamate autenticate devono usare `withCredentials: true` (Angular `HttpClient`) o `credentials: 'include'` (fetch). Senza, il cookie non viene inviato e la risposta è `401`.
>
> ⚠️ **Produzione**: i cookie hanno attualmente `secure=False`. Va portato a `True` con HTTPS attivo. *(Voce di backlog, non ancora risolta.)*

2.2 Livelli di accesso

Livello	Dipendenza	Comportamento
Pubblico	nessuna	Chiunque, anche non registrato
Autenticato	`Depends(get_current_user)`	Cookie `access_token` valido; altrimenti `401`
Amministrativo	`Depends(is_admin_user)`	Utente con `role = admin`; altrimenti `403`

`is_admin_user` è un `RoleChecker(allowed_roles=[UserRole.admin])`.

2.3 Cosa ha aggiunto il modulo Booking

Nessuna modifica al meccanismo di autenticazione esistente. Il modulo introduce però un secondo sistema di credenziali, **indipendente e non sostitutivo**:

- **Token monouso di prenotazione** — consentono a un ospite *non registrato* di confermare o cancellare la propria prenotazione senza avere un account. Non sono sessioni: valgono per una sola operazione su una sola prenotazione.

I due sistemi non si incrociano: un token di prenotazione non autentica l'utente, e una sessione valida non sostituisce il token nel link ricevuto via email.

---

3. Meccanismi trasversali introdotti

Questa sezione descrive tutto ciò che è stato aggiunto al progetto a supporto del modulo Booking.

3.1 Rate limiting

File: `src/security/rate_limiter.py`

Finestra scorrevole: per ogni chiave si conservano gli istanti delle richieste recenti e si scartano quelle uscite dalla finestra. A differenza del conteggio a finestra fissa, impedisce il raddoppio del limite a cavallo di due intervalli.

Limiti attivi (configurabili da `.env`):

Ambito	Limite	Finestra	Chiave
`GET /availability`	20	1 minuto	IP
`GET /occupancy`	20	1 minuto	IP
`POST /quote`	20	1 minuto	IP
`POST /` (creazione)	5	1 ora	IP
`POST /` (creazione)	3	24 ore	email
`POST /confirm`, `/cancel`	10	1 ora	IP
`POST /lookup`	10	1 ora	IP

I limiti per IP ed email sono complementari: un attaccante con molti indirizzi IP resta vincolato sul numero di prenotazioni riconducibili alla stessa email, e viceversa.

Risposta al superamento: `429` con header `Retry-After` in secondi.

```
HTTP/1.1 429 Too Many Requests
Retry-After: 42

{"message": "Troppe richieste: riprova più tardi"}
```

Due limiti noti, da tenere presenti:

1. **Il contatore è per processo.** Con `uvicorn --workers 4` ogni worker ha il proprio, quindi il limite effettivo è il quadruplo. Su un singolo worker il conteggio è esatto. L'interfaccia `RateLimitBackend` consente di sostituire lo storage con Redis senza toccare i router.
2. **`X-Forwarded-For` è ignorato per impostazione predefinita** (`trusted_proxy_count = 0`). Quell'header lo scrive il client: fidarsene senza un proxy che lo riscriva permetterebbe a chiunque di cambiare IP a ogni richiesta e azzerare il rate limiting. Quando l'app andrà dietro Nginx o Cloudflare, impostare `TRUSTED_PROXY_COUNT=1`.

3.2 Captcha — Cloudflare Turnstile

File: `src/security/captcha.py`

Verificato solo su `POST /api/v1/bookings/` (creazione pubblica). Scelto al posto di reCAPTCHA perché non profila gli utenti e non richiede un consenso ai cookie aggiuntivo.

- Disattivato per impostazione predefinita (`CAPTCHA_ENABLED=false`): in sviluppo non serve configurare nulla.
- Il frontend inserisce il widget Turnstile e invia il token risultante nel campo `captcha_token`.
- Token assente o rifiutato → `422`.

Comportamento in caso di disservizio: se Cloudflare non risponde o va in timeout (5 secondi), la richiesta **viene lasciata passare** e l'evento registrato come warning. Bloccare tutte le prenotazioni durante un disservizio di terze parti costerebbe incassi reali; le poche richieste automatiche che passassero creerebbero prenotazioni non confermate, che scadono da sole in quindici minuti.

3.3 Honeypot

Il campo `website` di `GuestBookingCreateSchema` è una trappola: va reso **invisibile via CSS** nel form e mai compilato da un utente reale. Se arriva valorizzato, la richiesta è automatizzata e viene respinta con `422`.

Costa nulla e intercetta i bot che compilano ogni campo del form.

3.4 Preventivo firmato (`quote_token`)

File: `src/security/quote_token.py`

Il client non invia mai un prezzo. Il flusso pubblico è a due passi per costruzione:

1. `POST /quote` calcola il totale e lo restituisce racchiuso in un JWT firmato, valido 15 minuti;
2. `POST /` accetta esclusivamente quel token.

Il token contiene camere, date, numero di ospiti, opzione di pagamento e totale. Protegge da due cose: la **manomissione del prezzo**, perché la firma lo rende immodificabile, e lo **spostamento della selezione**, perché non si possono sostituire le camere dopo il calcolo.

È firmato con lo stesso segreto dell'autenticazione ma con un claim `type` diverso: un access token non può essere speso come preventivo, né viceversa.

Difesa in profondità: anche con un token valido, il server **ricalcola** il totale alla creazione e rifiuta con `422` se diverge. La firma dice "questo prezzo l'ho emesso io", non "questo prezzo è ancora corretto".

3.5 Token monouso di prenotazione

File: `src/security/booking_tokens.py`

Generati con `secrets.token_urlsafe(32)` — 256 bit di entropia. Nel database finisce **solo l'hash SHA-256**, mai il valore in chiaro: una compromissione del database non consente di confermare o cancellare prenotazioni altrui.

Scopo	Uso
`CONFIRM_EMAIL`	Conferma della prenotazione (solo `PAY_ON_ARRIVAL`)
`MANAGE`	Gestione della prenotazione
`CANCEL`	Cancellazione

Sono **monouso**: al primo utilizzo `used_at` viene valorizzato e i tentativi successivi respinti. *Utilizzo* significa l'operazione che il token autorizza — confermare, annullare. **Leggere non è utilizzare**: `POST /manage` accetta il token `MANAGE` senza consumarlo, perché la pagina di gestione si ricarica (endpoint 5b).

Validità.

Scopo	Scade	Perché
`CONFIRM_EMAIL`	Insieme al blocco dello slot (`hold_expires_at`)	Confermare dopo che le date sono tornate disponibili non avrebbe senso
`MANAGE`	Alla **mezzanotte del check-in**, con tetto `MANAGE_TOKEN_MAX_DAYS` (400 gg)	Dal check-in in poi non consente più nulla: la cancellazione richiede lo stato `CONFIRMED`, e registrato l'arrivo la prenotazione è `CHECKED_IN`

Il tetto entra in gioco solo per soggiorni prenotati con grandissimo anticipo: un link valido per anni è un link che prima o poi finisce altrove.

Ritenzione. Una riga scaduta resta in tabella per `TOKEN_RETENTION_DAYS` (30 giorni), poi viene eliminata dallo sweeper. Non è sicurezza — un token scaduto non è già più spendibile, la scadenza viene verificata a ogni uso — ma è la sola informazione che quella riga conserva: *se* e *quando* un token era stato emesso, che è ciò che serve a rispondere all'ospite che scrive "il link non mi è mai arrivato". In tabella c'è l'hash, non il valore, quindi nessuno può rimandargli lo stesso link.

> ⚠️ **Il token viaggia nel body, mai in query string.** Un token nell'URL finisce negli access log, nell'header `Referer` e nella cronologia del browser. Il link nell'email punta al **frontend** (`{frontend_base_url}/booking/confirm?token=...`), e la SPA lo inoltra al backend con una `POST`.

3.6 Esposizione temporanea del token in sviluppo

Finché l'invio email non esiste (Step F), `POST /` restituisce il token di conferma nel campo `confirmation_token`, **ma solo quando** `EMAIL_ENABLED=false`. Attivando l'invio email il campo torna `null` automaticamente.

C'è un test che lo verifica, così l'affordance si spegne da sé senza che nessuno debba ricordarsene.

3.7 Gestione centralizzata degli errori

File: `src/exception/exception_handler.py`

Un **unico handler** registrato su `AppException` copre tutte le eccezioni di dominio, presenti e future: Starlette risolve percorrendo l'MRO dell'eccezione. Aggiungere un nuovo errore richiede una sola classe, non anche un handler.

Violazioni di vincolo del database *(dal 09/10/2026)*. Un handler su `IntegrityError` fa da rete di sicurezza per i percorsi che non traducono la violazione in un errore di dominio. Il riconoscimento (codice SQLSTATE) è in un modulo unico, `src/data/integrity_errors.py`, usato anche dai Service. Le violazioni dovute allo stato dei dati (sovrapposizione, duplicato, riferimento mancante) sono conflitti `409`; le altre sono bug del server, `500`. Il messaggio originale dell'errore **non viene mai restituito né registrato**: contiene l'istruzione SQL e i suoi parametri, cioè i dati dell'ospite. Nei log finiscono solo SQLSTATE, metodo e percorso.

Correzione importante introdotta allo Step B: gli errori di validazione producevano un `500` invece di un `422`, perché Pydantic v2 inserisce l'**oggetto eccezione vivo** nella chiave `ctx` e `json.dumps` non sa serializzarlo. Ora il contenuto di `ctx` viene convertito in stringa e le chiavi `input` e `url` rimosse.

3.8 Email transazionali

Cinque messaggi, ciascuno in versione HTML e testo semplice.

Template	Quando parte	Cosa contiene
`booking_pending`	Creazione con conferma richiesta	Link di conferma, scadenza del blocco
`booking_confirmed`	Conferma, o creazione già confermata	Riepilogo, **link di gestione**, termine gratuito, avviso di non rimborsabilità per `PAY_NOW`
`booking_cancelled`	Annullamento, dall'ospite o dal back-office	Riepilogo, importo già pagato se `PAID`, chiusura diversa se ha annullato la struttura
`booking_expired`	Sweeper	Avviso, invito a riprenotare

Il testo segue le regole, non le opzioni *(dal 28/09/2026)*:

- «già saldato» e «importo già pagato» dipendono da `payment_status == PAID`, non da `payment_option`: una `PAY_ON_ARRIVAL` incassata al banco è saldata come una pagata online;
- il link di gestione si presenta come «Gestisci la prenotazione» e promette l'annullamento **solo** se esiste un termine di cancellazione gratuita. La modifica non viene mai menzionata: non esiste per l'ospite;
- `PAY_NOW` riceve la riga «La tariffa con pagamento anticipato non è rimborsabile»;
- un termine alla mezzanotte locale si scrive «entro le 23:59 del *giorno prima*» (filtro `scadenza`): «fino al 14/11 alle 00:00» veniva letto come un giorno in più. Il termine calcolato non cambia;
- l'annullamento disposto dal back-office (`by_structure=True`) dice «L'annullamento è stato registrato dalla struttura» invece di «Non sei stato tu ad annullare?». La motivazione inserita dall'admin non viene mai mostrata all'ospite.

I link puntano alla SPA, non al backend: `{FRONTEND_BASE_URL}/prenotazione/conferma?token=...` e `/prenotazione/gestisci?token=...`. Il frontend legge il token dalla query string e lo inoltra nel **body di una POST** al backend. Il token non deve mai comparire in un URL del backend, dove finirebbe negli access log e nell'header `Referer`.

L'invio avviene dopo il commit, tramite `BackgroundTasks`. Non è un dettaglio implementativo: spedire dentro la transazione significherebbe, in caso di rollback, annunciare all'ospite una prenotazione che non esiste — e quella email non si richiama indietro.

Un invio fallito non fa fallire la richiesta. Se il server SMTP è irraggiungibile la prenotazione resta valida e l'evento finisce nei log. Stessa logica del captcha: rifiutare prenotazioni durante un disservizio di terze parti costerebbe incassi veri.

Nei log finiscono codice prenotazione, template, esito e destinatario mascherato (`m***@example.com`). **Non** il token, **non** il corpo del messaggio, **non** l'indirizzo completo.

Escaping. I template HTML hanno l'autoescape attivo. Il nome dell'ospite arriva da un form pubblico e finisce in un messaggio spedito con il mittente della struttura: senza escaping, chiunque potrebbe iniettare markup e link in una email che *sembra* del B&B.

Con `EMAIL_ENABLED=false` i messaggi vengono scritti nei log invece che spediti, link compreso. Perché siano visibili serve che i log applicativi abbiano una destinazione: `configure_logging()` in `main.py` attacca un handler al sottoalbero `src`. Senza, uvicorn configura solo i propri logger e ogni riga del progetto viene scartata in silenzio — il canale console sembrerebbe non funzionare. È l'unico punto del sistema in cui un token in chiaro finisce in un log, ed è attivo esattamente quando le email sono spente, cioè in sviluppo.

Nessuna email su `PENDING_PAYMENT`. Una prenotazione `PAY_NOW` nasce in attesa di incasso: l'ospite è ancora sulla pagina di pagamento e non c'è nulla da comunicargli. La notifica parte quando il webhook Stripe accerta l'incasso.

Un quinto template, `booking_slot_lost`, è arrivato con lo Step G: le camere vendute ad altri durante il pagamento. Merita un messaggio suo perché deve dire con chiarezza che **non c'è stato alcun addebito** — l'annullamento generico, che per il pagamento online parla di "importo già pagato", spaventerebbe l'ospite per un prelievo mai avvenuto.

3.9 Sweeper delle scadenze

Porta a `EXPIRED` le prenotazioni temporanee il cui blocco è scaduto. Parte con l'applicazione (`lifespan`) e gira ogni `SWEEPER_INTERVAL_SECONDS` (**300**, cinque minuti) su lotti di `SWEEPER_BATCH_SIZE`.

Perché cinque minuti e non uno, e perché non un'ora. La query costa poco, ma ogni passata interroga Stripe per ogni autorizzazione ancora viva: a sessanta secondi lo farebbe sessanta volte l'ora senza che nulla sia cambiato. Non si allunga oltre perché un pagamento abbandonato lascia su Stripe un intent **ancora pagabile** — finché non lo

annulliamo, l'ospite che ritrova la scheda aperta può completarlo su una prenotazione che noi consideriamo persa. Questo intervallo è la durata di quella finestra.

Pulizia dei token. Lo stesso ciclo elimina i token scaduti da più di `TOKEN_RETENTION_DAYS`, ma con cadenza propria: una volta ogni `TOKEN_PURGE_INTERVAL_HOURS` (24), a lotti di `TOKEN_PURGE_BATCH_SIZE` (1000). È manutenzione, non correttezza — farla a ogni passata sarebbe una `DELETE` ogni cinque minuti per liberare quasi sempre zero righe. Un suo fallimento viene registrato e non interrompe la passata: liberare gli slot non deve dipendere dalla pulizia di una tabella.

È l'unico proprietario della transizione a `EXPIRED`. In nessun altro punto del codice una prenotazione diventa `EXPIRED`.

Il sistema resta corretto anche se non gira. Gli slot tornano prenotabili per altre due vie: le query di disponibilità scartano i pending con blocco scaduto, e la *just-in-time expiration* disattiva le righe camera durante la creazione successiva sulle stesse date. Senza sweeper mancano la pulizia degli stati e l'avviso all'ospite, non la correttezza.

Le autorizzazioni si rilasciano prima di liberare. Da quando esistono i pagamenti, una prenotazione in attesa può avere un'autorizzazione viva su Stripe: lo sweeper la annulla **prima** di rimettere in vendita lo slot, e se non ci riesce non libera nulla. Dettagli in `payment_api.md` §6.

Con più worker uvicorn ne parte uno per processo. Le passate si dividono il lavoro grazie a `FOR UPDATE ... SKIP LOCKED`: chi arriva secondo salta le righe già prese in carico invece di aspettarle.

Arretrati. Se lo sweeper resta fermo a lungo, al riavvio sistema tutti gli stati ma **avvisa solo** le scadenze più recenti di `SWEEPER_NOTIFY_MAX_AGE_HOURS` (default 24). Spedire centinaia di email su richieste dimenticate da giorni è un modo efficace per farsi segnalare come spam.

Disattivabile con `SWEEPER_ENABLED=false`, per pilotarlo da uno scheduler esterno tramite l'endpoint 17.

---

4. Endpoint

4.1 Prospetto

#	Metodo	Rotta	Accesso	Rate limit	Captcha
1	`GET`	`/api/v1/bookings/availability`	Pubblico	20/min IP	no
1b	`GET`	`/api/v1/bookings/occupancy`	Pubblico	20/min IP	no
2	`POST`	`/api/v1/bookings/quote`	Pubblico	20/min IP	no
3	`POST`	`/api/v1/bookings/`	Pubblico	5/h IP + 3/g email	**sì**
4	`POST`	`/api/v1/bookings/confirm`	Pubblico	10/h IP	no
5	`POST`	`/api/v1/bookings/cancel`	Pubblico	10/h IP	no
5b	`POST`	`/api/v1/bookings/manage`	Pubblico	10/h IP	no
6	`POST`	`/api/v1/bookings/lookup`	Pubblico	10/h IP	no
7	`GET`	`/api/v1/bookings/me`	Autenticato	—	no
8	`POST`	`/api/v1/bookings/me`	Autenticato	—	no
9	`POST`	`/api/v1/bookings/me/{code}/cancel`	Autenticato	—	no

Area amministrativa, sotto `/api/v1/admin/bookings` — tutti con accesso **admin**:

#	Metodo	Rotta	Scopo
10	`GET`	`/`	Elenco filtrato e paginato
11	`GET`	`/{booking_id}`	Dettaglio completo con timeline
12	`POST`	`/`	Creazione per conto di terzi
~~13~~	~~`PATCH`~~	~~`/{booking_id}`~~	**Rimosso** — vedi la voce 13 più sotto
14	`POST`	`/{booking_id}/status`	Transizione di stato
15	`POST`	`/{booking_id}/payment`	Registrazione incasso manuale
16	`POST`	`/{booking_id}/extend-hold`	Proroga del blocco temporaneo
17	`POST`	`/sweep-expired`	Esecuzione immediata dello sweeper delle scadenze
18	`GET`	`/planning`	Tabellone: camere e soggiorni di un periodo *(dal 09/10/2026)*

1 · `GET /api/v1/bookings/availability`

Camere disponibili nell'intervallo richiesto.

Accesso	Pubblico
Rate limit	20 richieste/minuto per IP
Query	`check_in` (date, obbligatorio) · `check_out` (date, obbligatorio) · `guest_count` (int ≥ 1, default 1)
Risposta	`AvailabilityResponseSchema`

Codici: `200` · `422` date non valide · `429` limite superato

Logica. Recupera le camere abilitate e sottrae quelle occupate nell'intervallo. Una camera risulta occupata quando esiste una riga attiva con date sovrapposte e la prenotazione è in uno stato occupante — per gli stati temporanei, solo se il blocco non è ancora scaduto.

Non filtra per capienza, di proposito: nasconderebbe la possibilità di prenotare due camere doppie per quattro persone, che in un B&B è il caso normale delle famiglie. Restituisce invece tutte le camere libere con il flag `fits_all_guests`, più le combinazioni utili.

Le tre modalità di prenotazione sono servite da questa singola chiamata:

Modalità	Come ottenerla dal frontend
Camera singola sufficiente	Filtrare `rooms` su `fits_all_guests == true`
Scelta manuale	Presentare `rooms` per intero — la somma delle capienze è verificata al preventivo
Combinazioni suggerite	Usare `suggested_combinations`

Le combinazioni sono minimali: se togliendo una camera gli ospiti ci starebbero ancora, la proposta non viene restituita. Con una doppia, un'altra doppia e una quadrupla, per 4 persone si ottengono due proposte — la quadrupla da

sola, oppure le due doppie — e non le varianti ridondanti. Ordinate per numero di camere, poi prezzo, poi capienza sprecata. Massimo 10 proposte.

Test: `test_flusso_completo_availability_quote_create_confirm`, `test_availability_marca_le_camere_che_bastano_da_sole`, `test_availability_propone_combinazioni`, `test_availability_date_incoerenti_rifiutate`, `test_rate_limit_con_retry_after`, più l'intero file `test_availability_combinations.py` (11 test sulla logica pura).

```
curl -i "http://localhost:8000/api/v1/bookings/availability?check_in=2026-12-01&check_out=2026-12-03&g
```

1b · `GET /api/v1/bookings/occupancy`

Notti non disponibili per un insieme di camere — il calendario della modalità «Per camera».

Accesso	Pubblico (nessuna autenticazione)
Rate limit	20 richieste/minuto per IP
Query	`room_ids` (UUID, obbligatorio, **da ripetere** per ogni camera: `?room_ids=…&room_ids=…`) · `date_from` (date, obbligatorio) · `date_to` (date, obbligatorio)
Risposta	`OccupancyResponseSchema`

Codici: `200` · `404` camera inesistente · `409` camera disattivata · `422` parametri non validi · `429` limite superato

La finestra è `[date_from, date_to)`: `date_from` inclusa, `date_to` **esclusa**, come arrivo e partenza. Novembre e dicembre si chiedono con `date_from=2026-11-01&date_to=2027-01-01`.

Si ragiona per notti. La notte del giorno `d` è quella fra `d` e `d + 1`. Una prenotazione dal 12 al 14 occupa le notti del **12** e del **13**: il 14 l'ospite parte, e quel giorno un altro può arrivare. Per questo il 14 **non** compare fra le notti non disponibili.

Logica. Verifica le camere come la creazione (`load_bookable_rooms`: inesistente → `404`, disattivata → `409`), legge gli intervalli delle righe camera che occupano la finestra, li ritaglia sulla finestra e li trasforma in notti. Una notte è non disponibile se **almeno una** delle camere richieste è occupata. La definizione di "occupato" è **la stessa** di `/availability` (`BookingRepository._occupying_item_conditions`): riga attiva, stato occupante, blocco temporaneo non scaduto. Le due letture non possono quindi contraddirsi — lo verifica `test_coerente_con_la_ricerca_per_date`.

Cosa non restituisce, di proposito. Solo date, già unite fra le camere. Nessun codice, nome, durata, camera o stato delle singole prenotazioni: l'endpoint è pubblico e servirà anche alla prenotazione autonoma dell'ospite, a cui serve sapere *se* è libero, non *chi* occupa.

Vincoli (`OccupancyRequestSchema`, `422` altrimenti):

Regola	Messaggio
da 1 a 5 camere, senza duplicati	«Devi selezionare almeno una camera» · «…ID duplicati» · «…più di 5 camere…»
`date_to` > `date_from`	«La data finale deve essere successiva a quella iniziale»
`date_from` ≥ oggi (fuso della struttura)	«Il periodo non può iniziare nel passato»
finestra ≤ `occupancy_max_window_days` (**92**) giorni	«Il periodo richiesto non può superare 92 giorni»
`date_to` ≤ oggi + 365 + 30 giorni (anticipo massimo più soggiorno massimo)	«Il periodo richiesto va oltre il limite di prenotazione»

Il limite di 92 giorni impedisce di scaricare l'occupazione di un anno intero con una richiesta sola. Il calendario ne chiede due mesi alla volta.

Test: classe `TestOccupancy` in `test_occupancy_api.py` (18), `TestOccupancyRequest` in `test_booking_schema.py` (10), `TestExpandNights` in `test_availability_combinations.py` (4).

```
curl -i "http://localhost:8000/api/v1/bookings/occupancy?room_ids=<UUID1>&room_ids=<UUID2>&date_from=2
```

```
{
  "date_from": "2026-11-01",
  "date_to": "2027-01-01",
  "room_ids": ["<UUID1>", "<UUID2>"],
  "unavailable_nights": ["2026-11-12", "2026-11-13", "2026-12-24"]
}
```

---

2 · `POST /api/v1/bookings/quote`

Preventivo firmato per una selezione di camere.

Accesso	Pubblico
Rate limit	20 richieste/minuto per IP
Body	`BookingQuoteRequestSchema`
Risposta	`BookingQuoteResponseSchema`

Codici: `200` · `404` camera inesistente · `409` camera occupata o disabilitata · `422` date non valide, ospiti oltre la capienza · `429`

Logica. Carica le camere, verifica che esistano e siano abilitate, controlla che la somma delle capienze copra gli ospiti, verifica la disponibilità, calcola il prezzo ed emette il token firmato.

È **di sola lettura**: non blocca nulla e non crea righe. La disponibilità viene verificata per dare all'utente un errore immediato invece di farglielo scoprire dopo aver compilato i propri dati, ma **la garanzia resta al momento della prenotazione**: fra il preventivo e la conferma qualcun altro può sempre arrivare prima.

Lo sconto per pagamento online (10%) è applicato qui e incluso nel token.

Test: `test_flusso_completo_...`, `test_preventivo_su_camera_inesistente`, e l'intero `test_pricing_service.py` (18 test su calcoli, arrotondamento e token).

```
curl -i -X POST http://localhost:8000/api/v1/bookings/quote \
  -H "Content-Type: application/json" \
  -d '{"check_in":"2026-12-01","check_out":"2026-12-03","guest_count":2,
       "room_ids":["<UUID_CAMERA>"],"payment_option":"PAY_ON_ARRIVAL"}'
```

---

3 · `POST /api/v1/bookings/`

Crea una prenotazione come ospite non registrato.

Accesso	Pubblico
Rate limit	5/ora per IP **e** 3/giorno per email
Captcha	Sì (se `CAPTCHA_ENABLED=true`)
Body	`GuestBookingCreateSchema`
Risposta	`BookingCreatedSchema` — **201**

Codici: `201` · `404` camera inesistente · `409` slot occupato · `410` — · `422` preventivo scaduto o manomesso, honeypot, condizioni non accettate, captcha fallito · `429`

Logica — l'operazione più delicata del modulo. Tutto dentro una sola transazione:

1. verifica del `quote_token` (firma e scadenza);
2. caricamento camere, controllo esistenza e abilitazione;
3. controllo capienza totale ≥ ospiti;
4. **ricalcolo del prezzo** e confronto con il token → `422` se diverge;
5. controllo del numero di prenotazioni in attesa per quella email;
6. **lock pessimistico** sulle camere, ordinato per id (nessun deadlock possibile);
7. **just-in-time expiration**: libera gli slot il cui blocco è scaduto;
8. verifica disponibilità → `409` con l'elenco delle camere non più libere;
9. generazione del codice prenotazione univoco;
10. creazione della prenotazione e delle righe camera, con prezzo congelato;
11. `flush` → **qui scatta l'exclusion constraint del database**: se due richieste simultanee arrivano insieme, una sola passa;
12. riga di storico e token di conferma.

Stato iniziale:

Opzione di pagamento	Stato	Token di conferma
`PAY_ON_ARRIVAL`	`PENDING_CONFIRMATION`	sì
`PAY_NOW`	`PENDING_PAYMENT`	no — il pagamento è la verifica d'identità

Lo slot resta bloccato **15 minuti** (`hold_expires_at`). Scaduto quel termine senza conferma, torna prenotabile e lo sweeper porta la prenotazione a `EXPIRED`.

Email: `booking_pending` con `PAY_ON_ARRIVAL`. Con `PAY_NOW` **nessuna email**: l'ospite è ancora sulla pagina di pagamento, e la conferma parte quando il webhook Stripe accerta l'incasso — vedi `payment_api.md`.

Test: `test_flusso_completo_...`, `test_slot_occupato_risponde_409`, `test_la_creazione_manda_una_sola_email_di_conferma`, `test_il_pagamento_online_non_manda_ancora_nulla`, `test_un_canale_email_guasto_non_impedisce_la_prenotazione`, `test_honeypot_compilato_rifiutato`, `test_condizioni_non_accettate_rifiutate`, `test_token_di_conferma_non_esposto_con_email_attiva`, più i 14 test di `test_booking_service.py` fra cui **`test_due_prenotazioni_concorrenti_una_sola_vince`**, ripetuto 10 volte.

```
curl -i -X POST http://localhost:8000/api/v1/bookings/ \
  -H "Content-Type: application/json" \
  -d '{"quote_token":"<TOKEN_DAL_PREVENTIVO>",
       "guest":{"firstname":"Mario","lastname":"Rossi",
                "email":"mario@example.com","phone_number":"3331234567"},
       "accept_terms":true}'
```

---

4 · `POST /api/v1/bookings/confirm`

Conferma tramite il token ricevuto per email.

Accesso	Pubblico
Rate limit	10/ora per IP
Body	`BookingConfirmSchema`
Risposta	`BookingPublicSchema`

Codici: `200` · `400` token inesistente o già speso · `409` già confermata · `410` blocco scaduto · `429`

Logica. Cerca il token per hash, verifica che sia del tipo corretto e non usato, controlla che né il token né il blocco siano scaduti, esegue la transizione a `CONFIRMED`, azzera `hold_expires_at`, calcola il termine di cancellazione gratuita, marca il token come speso e invalida gli altri della stessa prenotazione.

Doppio clic sul link: se il token è già speso e la prenotazione è `CONFIRMED`, la risposta è `409` con *"La prenotazione è già stata confermata"* — distinto dal generico "link non valido", per non far dubitare l'ospite che la conferma sia andata a buon fine.

Blocco scaduto: `410`. Lo stato resta `PENDING_CONFIRMATION` finché non passa lo sweeper; lo slot è comunque già tornato prenotabile.

Emette il token di gestione. Alla conferma viene creato un token `MANAGE`, valido fino alla data di partenza, che viaggia nel link dell'email di riepilogo. Per un ospite non registrato è **l'unico modo di annullare in autonomia**: non ha un account, e il `lookup` è in sola lettura.

Email: `booking_confirmed`, con il link di gestione.

Test: `test_flusso_completo_...`, `test_doppia_conferma_risponde_409`, `test_conferma_completa_il_ciclo`, `test_doppia_conferma_segnalata`, `test_conferma_dopo_la_scadenza_rifiutata`, `test_la_conferma_manda_il_riepilogo_con_il_link_di_gestione`.

```
curl -i -X POST http://localhost:8000/api/v1/bookings/confirm \
  -H "Content-Type: application/json" -d '{"token":"<TOKEN_DI_CONFERMA>"}'
```

---

5 · `POST /api/v1/bookings/cancel`

Cancellazione tramite link ricevuto per email.

Accesso	Pubblico
Rate limit	10/ora per IP
Body	`BookingCancelSchema`
Risposta	`BookingPublicSchema`

Codici: `200` · `400` token non valido · `409` non cancellabile · `429`

Logica. Accetta token di tipo `CANCEL` o `MANAGE`. Verifica che la cancellazione sia ammessa:

Situazione	Esito
Stato `PENDING_*`	Consentita
`CONFIRMED` + `PAY_ON_ARRIVAL` entro il termine	Consentita, gratuita
`CONFIRMED` + `PAY_ON_ARRIVAL` oltre il termine	`409` — invito a contattare la struttura
`CONFIRMED` + `PAY_NOW`	`409` — non rimborsabile, invito a contattare la struttura
Stato terminale	`409`

Alla cancellazione lo slot torna **immediatamente** prenotabile, e il token viene speso: un secondo tentativo con lo stesso link risponde `400`.

> **Quello che questa tabella non dice, e conviene sapere.** Una disdetta tardiva o una mancata presentazione su `PAY_ON_ARRIVAL` **non costano nulla all'ospite**. Non c'è penale, non c'è carta a garanzia, e l'unico effetto pratico del `409` oltre il termine è che l'ospite non riesce ad annullare da solo: deve telefonare, e finché un admin non interviene lo slot resta occupato. Per la struttura è il caso peggiore — una camera invenduta all'ultimo momento e nessun deterrente. `PricingService.compute_cancellation_penalty` esiste già ma **non è chiamata da nessuna parte**. La soluzione (carta raccolta alla prenotazione con mandato SEPA/SCA, penale addebitata off-session) è tracciata come **debito tecnico #22**: non è una riga da aggiungere qui, richiede un `SetupIntent`, il consenso esplicito dell'ospite e la gestione dell'addebito rifiutato.
>
> **Prima di chiamare questo endpoint, chiamare il 5b.** La tabella qui sopra il frontend non deve indovinarla: `POST /manage` restituisce `can_cancel` e `blocked_by` calcolati sulla prenotazione reale, in questo istante, dalla stessa funzione che decide qui.

Verifica della scadenza. Un token scaduto viene respinto con `400`. Non era vero fino al 26/09/2026: questo endpoint controllava che il token esistesse e non fosse già speso, **ma non la sua scadenza**. Con il `MANAGE` che scade all'arrivo, la finestra non era teorica. Oggi entrambi gli endpoint passano dallo stesso `_load_guest_token`.

Da dove arriva il token. È il `MANAGE` emesso alla conferma (endpoint 4) o alla creazione di una prenotazione che nasce già confermata (endpoint 12). Arriva all'ospite nel link *"Gestisci o annulla la prenotazione"* dell'email di riepilogo.

> ⚠️ **Fino allo Step F questo endpoint era irraggiungibile.** Cercava un token `CANCEL` o `MANAGE`, ma nessun percorso di codice ne emetteva uno: la rotta era scritta, testata a livello di servizio e documentata, e nessun client poteva procurarsi la credenziale richiesta. Il difetto è sopravvissuto perché i test del servizio costruivano il token a mano — cosa che un ospite non può fare. Vedi §8bis.11.3.

Email: `booking_cancelled`.

Test: `test_slot_liberato_dopo_la_cancellazione`, `test_il_link_ricevuto_per_email_permette_davvero_di_annullare`, `test_il_link_di_gestione_si_spende_una_volta_sola`, `test_l_annullamento_avvisa_l_ospite`.

5b · `POST /api/v1/bookings/manage`

Legge una prenotazione dal link di gestione, senza consumarlo.

Accesso	Pubblico
Rate limit	10/ora per IP (lo stesso di `/lookup`)
Body	`BookingManageRequestSchema` — solo `token`
Risposta	`BookingManageSchema`

Codici: `200` · `400` token non valido, già speso o scaduto · `422` token di forma errata · `429`

Perché esiste. Il link nell'email porta l'ospite su `{frontend}/prenotazione/gestisci?token=...`, cioè sulla SPA **con un token e nient'altro**. Prima di questo endpoint quella pagina non aveva modo di mostrargli la prenotazione: `POST /lookup` chiede codice ed email, che il link non contiene, e `POST /cancel` accetta il token ma annulla. L'unica pagina costruibile era un pulsante *"Annulla"* cieco.

È la stessa forma del difetto dello Step F (§8bis.11.3): una credenziale emessa, recapitata, e senza un endpoint che le permetta di fare ciò per cui l'ospite la riceve.

Il token non viene speso. La pagina si ricarica, e un token consumato alla prima lettura lascerebbe l'ospite davanti a un errore al primo aggiornamento — senza più modo di annullare. Resta spendibile finché `POST /cancel` non lo usa davvero.

Accetta `MANAGE` e `CANCEL`, come `/cancel`: chi può annullare può a maggior ragione leggere ciò che sta per annullare. Un token `CONFIRM_EMAIL` viene invece respinto.

Cosa restituisce oltre alla prenotazione. La politica di cancellazione **applicabile in questo momento**, così la pagina dice all'ospite cosa succederà *prima* che clicchi invece di farglielo scoprire con un `409`:

Campo	Significato
`can_cancel`	`true` se `POST /cancel` andrebbe a buon fine adesso
`blocked_by`	`STATUS_NOT_CANCELLABLE` · `NON_REFUNDABLE` · `DEADLINE_PASSED` — solo quando `can_cancel` è `false`
`message`	Lo stesso testo che `POST /cancel` restituirebbe nel `409`, già scritto per l'ospite
`free_until`	Termine di cancellazione gratuita, quando esiste

> **Il frontend usi `blocked_by`, non `message`.** Il codice è stabile e permette di decidere *cosa mostrare* — un avviso, un numero di telefono, un pulsante disabilitato. Il testo è pronto all'uso e allineato a quello del `409`, ma resta una frase in italiano, e una frase non si interroga.

Non possono divergere. `can_cancel` e il comportamento di `/cancel` vengono dalla **stessa funzione** — `BookingService.describe_guest_cancellation` — perché due implementazioni, una che decide e una che spiega,

si sarebbero separate alla prima modifica della politica commerciale, e in silenzio: la pagina avrebbe annunciato *"annulla gratuitamente"* e l'endpoint avrebbe risposto `409`. C'è un test che percorre la catena da un capo all'altro (`test_la_politica_annunciata_e_quella_applicata`).

Dopo l'annullamento il token è speso, quindi anche la lettura si chiude con `400`. È deliberato: la risposta di `POST /cancel` porta già la prenotazione annullata, e la SPA ha tutto per mostrare l'esito senza rileggere.

Test: `test_il_link_di_gestione_mostra_la_prenotazione`, `test_la_lettura_non_consuma_il_token`, `test_la_politica_annunciata_e_quella_applicata`, e altri cinque.

---

6 · `POST /api/v1/bookings/lookup`

Consulta una prenotazione con codice ed email.

Accesso	Pubblico
Rate limit	10/ora per IP
Body	`BookingLookupSchema`
Risposta	`BookingPublicSchema`

Codici: `200` · `404` nessuna corrispondenza · `429`

Logica. Richiede codice **e** email: il solo codice non basta, per evitare che tentativi a forza bruta espongano dati di altri ospiti. Il confronto sull'email è insensibile alle maiuscole; il codice viene normalizzato (spazi rimossi, maiuscolo).

Il `404` ha un messaggio **generico e identico** in ogni caso di fallimento: non deve lasciar capire se il codice esista e sia soltanto l'email a non corrispondere.

Test: `test_lookup_con_dati_errati_non_rivela_nulla`, `test_lookup_con_codice_ed_email_corretti`.

> ⚠️ **Gli endpoint 7, 8 e 9 sono attualmente disattivati nel sorgente** (`src/routers/booking_router.py`: il blocco è racchiuso in una stringa, quindi il codice esiste ma non viene registrato). Chiamarli risponde `404`, non `401`: il percorso non esiste proprio.
>
> Restano documentati perché il codice c'è e tornerà quando esisterà l'autenticazione dell'ospite finale. I cinque test che li coprono sono marcati `skip` in `test_booking_api.py` con la stessa motivazione — togliere il marcatore è tutto ciò che servirà per sapere se funzionano ancora. **Debito tecnico #23.**

7 · `GET /api/v1/bookings/me`

Le prenotazioni dell'utente autenticato.

Accesso	**Autenticato** (cookie `access_token`)
Query	`limit` (1–100, default 20) · `offset` (≥ 0, default 0)
Risposta	`List[BookingPublicSchema]`

Codici: `200` · `401` sessione assente o scaduta

Ordinate per data di arrivo decrescente.

Test: `test_le_mie_prenotazioni_richiedono_autenticazione`.

---

8 · `POST /api/v1/bookings/me`

Crea una prenotazione come utente autenticato.

Accesso	Autenticato
Body	`UserBookingCreateSchema`
Risposta	`BookingCreatedSchema` — **201**

Codici: `201` · `401` · `404` · `409` · `422`

Logica. Identica alla creazione pubblica, con tre differenze: nessun captcha e nessun limite per email (la sessione autenticata è già una barriera contro l'automazione), e l'anagrafica **copiata dal profilo** invece che richiesta.

Lo snapshot anagrafico resta comunque congelato: una modifica successiva all'account non altera la prenotazione.

Test: `test_creazione_utente_richiede_autenticazione`, `test_prenotazione_utente_autenticato_viene_persistita`.

> Quest'ultimo rilegge la prenotazione in una **richiesta successiva**, non solo nel corpo della risposta: è l'unico modo per accorgersi di un mancato commit. Vedi §11.2.

---

9 · `POST /api/v1/bookings/me/{code}/cancel`

Cancella una propria prenotazione.

Accesso	Autenticato
Path	`code` — codice prenotazione, es. `BB-2026-A7K3QX`
Body	`OwnBookingCancelSchema` — solo `reason` opzionale, nessun token
Risposta	`BookingPublicSchema`

Codici: `200` · `401` · `404` inesistente **o di un altro utente** · `409` non cancellabile

> **Si identifica col codice, non con l'`id`.** `BookingPublicSchema` non espone l'identificativo interno, quindi un client non avrebbe modo di procurarselo: il codice è l'unico riferimento pubblico di una prenotazione, ed è quello che l'utente legge nella conferma.
>
> **Perché `404` e non `403`** su una prenotazione altrui: distinguere i due casi confermerebbe l'esistenza del codice e consentirebbe di sondare le prenotazioni di altri utenti.

Test: `test_utente_cancella_la_propria_prenotazione`, `test_non_si_cancella_la_prenotazione_di_un_altro`.

4bis. Endpoint amministrativi

Tutte le rotte sotto `/api/v1/admin/bookings` richiedono un utente con `role = admin`. Senza cookie di sessione rispondono `401`, con un utente `role = user` rispondono `403`.

---

10 · `GET /api/v1/admin/bookings/`

Elenco filtrato e paginato.

Accesso	Amministrativo
Query	`status` (ripetibile) · `date_from` · `date_to` · `email` · `code` · `room_id` · **`sort`** · `page` (default 1) · `page_size` (default 20, max 100)
Risposta	`PaginatedBookingsSchema`

Codici: `200` · `401` · `403` · `422` intervallo di date incoerente, `page_size` oltre il limite, `sort` sconosciuto

Ordinamento *(dal 01/10/2026)*. `sort` vale `CHECK_IN_DESC` (predefinito: arrivo più lontano per primo), `CHECK_IN_ASC` (arrivo più vicino per primo) o `CREATED_DESC` (ultime inserite per prime). L'ultimo criterio è sempre l'id, così una prenotazione non compare su due pagine diverse.

Logica. `date_from` seleziona i soggiorni che **terminano dopo** quella data e `date_to` quelli che **iniziano prima**: insieme individuano i soggiorni che si sovrappongono all'intervallo, non solo quelli interamente contenuti.

Il filtro `status` si ripete per selezionarne più di uno: `?status=CONFIRMED&status=CHECKED_IN`.

Le righe camera sono caricate con `selectinload`: una query aggiuntiva per l'intera pagina, non una per prenotazione.

Test: `TestRead::test_elenco_paginato`, `test_filtro_per_stato`, `test_filtro_per_email`, `test_intervallo_di_date_invertito_rifiutato`.

```
curl -i -b cookies.txt "http://localhost:8000/api/v1/admin/bookings/?status=CONFIRMED&page=1&page_size
```

---

11 · `GET /api/v1/admin/bookings/{booking_id}`

Dettaglio completo con timeline degli stati.

Accesso	Amministrativo
Risposta	`BookingSchema`

Codici: `200` · `401` · `403` · `404`

Logica. Vista completa: audit, canale di origine, note interne, riferimenti di pagamento e `status_history` con ogni transizione, attore e motivazione.

> Il campo **`version`** è il contatore di optimistic locking del database. Nessuna rotta lo accetta più in ingresso — la modifica è stata rimossa, vedi la voce 13 — ma resta esposto perché è il valore che un client dovrà rimandare quando quel percorso verrà riscritto.

Test: `TestRead::test_dettaglio_con_storico`, `test_dettaglio_inesistente`.

12 · `POST /api/v1/admin/bookings/`

Creazione per conto di terzi.

Accesso	Amministrativo
Body	`AdminBookingCreateSchema`
Risposta	`AdminBookingCreatedSchema` — **201**

Codici: `201` · `401` · `403` · `404` utente o camera inesistenti · `409` slot occupato · `422` validazione

Logica. L'intestatario è un utente registrato (`user_id`) **oppure** un profilo inserito a mano (`guest`), mai entrambi né nessuno dei due.

> **In pratica, usare `guest`.** Il ramo `user_id` funziona ed è pronto, ma oggi non porta alcun vantaggio: l'unico posto in cui una prenotazione legata a un account si vede è `GET /bookings/me`, che richiede l'autenticazione dell'ospite finale — un percorso che al momento non esiste (le rotte `/me` sono disattivate nel sorgente). Non è un rischio, è semplicemente inutile finché quel percorso non c'è.
>
> **Le difese del canale pubblico qui non si applicano**, di proposito: niente captcha, niente limite di 3 prenotazioni in sospeso per email, niente rate limit per indirizzo (`enforce_pending_limit=False`). L'admin è un attore fidato e la rotta è già protetta da `is_admin_user` a livello di router. Vale la pena saperlo perché significa che una prenotazione creata da qui **non** ha attraversato nessuno dei controlli anti-abuso descritti al §3.

Con `skip_email_confirmation` attivo — il default — la prenotazione nasce già `CONFIRMED`, senza token e senza blocco temporaneo: è il caso della prenotazione telefonica, dove l'identità è già stata verificata parlando con l'ospite. Disattivandolo si ottiene il flusso normale con conferma via email.

Non serve un preventivo firmato: l'admin è un attore fidato e il prezzo è calcolato dal server. Il percorso di creazione è però **lo stesso del canale pubblico** — lock, liberazione hold scaduti, verifica, exclusion constraint — quindi anche l'admin non può creare overbooking.

Sono consentite **date nel passato**, per registrare a posteriori un walk-in o correggere un errore.

Coerenza del pagamento *(dal 28/09/2026)*. Una prenotazione nata dal back-office non passa mai da Stripe: ciò che non si incassa qui non verrà incassato altrove. Le combinazioni che la lascerebbero impossibile da completare rispondono `422`:

Richiesta	Perché è rifiutata
`payment_method = STRIPE_CARD`	Un pagamento online lo registra solo il webhook
`payment_method` senza `mark_as_paid`	Il campo descrive come l'ospite **ha già pagato**, non come pagherà
`mark_as_paid` senza `payment_method`	Un incasso ha sempre un mezzo
`mark_as_paid` con `skip_email_confirmation = false`	Se la conferma non arrivasse, l'incasso resterebbe su una prenotazione `EXPIRED`
`payment_option = PAY_NOW` senza `mark_as_paid`	Nata `CONFIRMED`, non potrebbe più essere pagata online (`POST /payments/intent` la rifiuta come già confermata): resterebbe scontata del 10% e mai saldata

> ⚠️ **Frontend**: quando la data di arrivo è precedente a oggi, mostrare un dialog di conferma esplicito prima di inviare. Il backend lo consente di proposito, quindi l'unica difesa contro il refuso di digitazione è quell'avviso.

Email: `booking_confirmed` con il link di gestione quando nasce già confermata, `booking_pending` quando `skip_email_confirmation` è disattivato. Una prenotazione presa al telefono resta una prenotazione di cui l'ospite deve avere traccia scritta.

Test: l'intera classe `TestCreate` (7 test); le combinazioni di pagamento in `test_booking_schema.py::TestAdminBookingCreate`.

---

13 · `PATCH /api/v1/admin/bookings/{booking_id}` — RIMOSSO

Questa rotta non esiste più. Un `PATCH` su questo percorso risponde `405`.

Cosa faceva. Modificava date, camere, ospiti, anagrafica e note, ricalcolando il prezzo e ricostruendo le righe camera. Aveva optimistic locking lato client (`version` → `409`) e sapeva escludere la prenotazione da se stessa nel calcolo dei conflitti, così che allungarla di un giorno non la facesse collidere con le proprie righe.

Perché è stata rimossa. La prenotazione registra **quanto è dovuto** (`total_price`), non **quanto è stato incassato**: non esiste una colonna `amount_paid`. Finché nulla cambia i due valori coincidono e la mancanza non si nota. Ma quando l'admin spostava le date di una prenotazione già saldata e il totale passava da 180 a 240, il database finiva per dire `total_price = 240` con `payment_status = PAID`, e **l'informazione che l'ospite aveva pagato 180 spariva**. Non si poteva più calcolare né un conguaglio né un rimborso, e l'unico modo di recuperare l'importo reale era chiederlo a Stripe.

Il difetto non era nella rotta ma nel modello, che confonde il dovuto con l'incassato. Rimuovere la rotta è la soluzione onesta finché quel modello non cambia: meglio nessuna modifica che una modifica che distrugge in silenzio un dato contabile.

Cosa fare al suo posto. Annullare la prenotazione (endpoint 14, `new_status = CANCELLED`) e ricrearla (endpoint 12). Si perde il codice prenotazione e si spezza lo storico, ma nessun dato viene falsato. Su una prenotazione già incassata, il rimborso o il conguaglio si concordano con l'ospite e si registrano con l'endpoint 15.

Cosa serve per riaverla, in ordine: una colonna `amount_paid` con la relativa migrazione; il calcolo esplicito della differenza fra dovuto e incassato; una decisione su cosa fare quando la differenza è a favore dell'ospite (rimborso automatico? nota di credito? nulla?); e l'email che comunica la modifica, perché una prenotazione che cambia prezzo senza che l'ospite lo sappia è un reclamo garantito. **Debito tecnico #21.**

Cosa resta in piedi. Il campo `version` è ancora esposto da `BookingSchema` — è il contatore di optimistic locking del database — e l'eccezione `ConcurrentModification` esiste ancora ma nessun percorso la solleva. Il parametro `exclude_booking_id` resta disponibile nel `BookingRepository`. Sono i pezzi che serviranno quando la modifica verra riscritta.

Test: `TestUpdateRimossa::test_patch_non_esiste_piu` verifica che la rotta risponda `405`. Verifica un'assenza, di proposito: una rotta si riaggiunge in tre righe, e un `PATCH` che torna a rispondere `200` senza che nessuno l'abbia deciso è esattamente il modo in cui un problema noto rientra dalla finestra.

---

14 · `POST /api/v1/admin/bookings/{booking_id}/status`

Transizione di stato.

Accesso	Amministrativo
Body	`BookingStatusUpdateSchema`
Risposta	`BookingSchema`

Codici: `200` · `401` · `403` · `404` · `409` transizione non ammessa o incoerente con le date, oppure camere rivendute (vedi sotto) · `422` motivazione mancante sull'annullamento o sulla partenza anticipata

Logica. Oltre alle transizioni della macchina a stati, valgono tre controlli temporali che intercettano i refusi più comuni del back-office:

Transizione	Vincolo
→ `CHECKED_IN`	non prima della data di arrivo
→ `NO_SHOW`	solo dal giorno **dopo** l'arrivo
→ `COMPLETED`	solo dal giorno **dopo** l'arrivo *(dal 07/10/2026; prima: non prima della partenza)*

Partenza anticipata *(dal 07/10/2026)*. Concludere prima della data di partenza è ammesso, ma con **motivazione obbligatoria** (`422` `StatusChangeReasonRequired` , *«Per concludere il soggiorno prima della data di partenza serve una motivazione»*): le notti rimaste restano **occupate e pagate**, ed è il caso che più facilmente genera una contestazione. Liberarle vorrebbe dire accorciare il soggiorno, cioè modificare la prenotazione (debito #21).

Mancata presentazione *(dal 07/10/2026)*. `NO_SHOW` **libera le camere**, come un annullamento: nessuno le usa, e le notti rimaste si possono rivendere. Anche le notti passate tornano libere (nessuno vi ha dormito), così si può registrare a posteriori un ospite arrivato senza prenotazione. Le prenotazioni già segnate prima di questa data vengono allineate dalla migrazione `a7c3e91f5d20` .

Due transizioni che la macchina a stati ammette sono **negate all'admin** *(dal 28/09/2026)*, perché appartengono a un altro attore:

Transizione	Risposta	Perché
→ `EXPIRED`	`409` *«La scadenza è gestita automaticamente dal sistema»*	È dello sweeper (piano del modulo, §4.1). Per liberare subito uno slot, l'admin annulla
`PENDING_PAYMENT` → `CONFIRMED`	`409` *«Questa prenotazione si conferma solo con il pagamento online»*	È del webhook. Confermata a mano resterebbe scontata e mai pagata

Conferma dopo la scadenza del blocco *(dal 09/10/2026)*. Una prenotazione `PENDING_CONFIRMATION` il cui blocco è scaduto, ma che lo sweeper non ha ancora portato a `EXPIRED` , non occupa più le notti: una nuova prenotazione le può prendere. Se l'admin la conferma dopo, riattivarla violerebbe il vincolo anti-overbooking. Prima la risposta era `500` ; ora è `409` `RoomNotAvailable` , *«Le camere sono state prenotate da qualcun altro nel frattempo: la prenotazione non si può più confermare»*. La transazione viene annullata: la prenotazione resta in attesa, nessuna voce di cronologia, nessuna email, e scadrà da sola.

L'annullamento libera **immediatamente** lo slot. Annullare una prenotazione `PENDING_PAYMENT` è sicuro anche se l'ospite è sulla pagina di pagamento: un'autorizzazione che arriva dopo viene **rilasciata** senza incasso (esito `NOT_PAYABLE` , vedi `payment_api.md`).

Email *(dal 04/10/2026)* — due transizioni avvisano l'ospite:

Transizione	Messaggio	Perché
→ `CANCELLED`	`booking_cancelled` (*«annullata dalla struttura»*)	Un annullamento deciso al banco l'ospite potrebbe non saperlo
`PENDING_CONFIRMATION` → `CONFIRMED`	`booking_confirmed` **con il link di gestione**	Come la conferma via email: è l'unico strumento con cui un ospite non registrato vede o annulla la prenotazione. Il vecchio link di conferma viene invalidato: chi lo apre dopo riceve `409` *«La prenotazione è già stata confermata»*, lo stesso messaggio del doppio clic. Se l'arrivo è già oggi o passato, il link non viene emesso (scadrebbe all'arrivo) e l'email parte senza

Arrivo, conclusione e mancata presentazione non mandano nulla: riguardano il funzionamento interno della struttura e l'ospite le conosce già perché era presente.

Risposta completa *(dal 04/10/2026)* — `status_history` contiene anche la voce appena scritta. Prima la cronologia veniva restituita come era stata letta *prima* della transizione, e il client doveva rileggere il dettaglio. Vale per tutte le risposte `BookingSchema` dell'area admin (creazione, stato, incasso, proroga).

Test: `TestOperations::test_annullamento_senza_motivazione_rifiutato`, `test_check_in_anticipato_rifiutato`, `test_transizione_illegale_rifiutata`, `test_annullamento_libera_lo_slot`, `test_scadenza_manuale_rifiutata`, `test_conferma_manuale_di_un_pagamento_online_rifiutata`, `test_conferma_manuale_di_una_attesa_email_consentita`; classe `TestStatusResponseAndEmails` (6): cronologia nella risposta di creazione e di cambio stato, link di gestione funzionante dopo la conferma dell'admin, vecchio link di conferma che risponde «già confermata», nessun link per un soggiorno già iniziato, nessuna email su arrivo e conclusione. Classe `TestEarlyDepartureAndNoShow` (6): partenza anticipata con e senza motivazione, rifiutata il giorno dell'arrivo, notti rimaste ancora occupate; non presentato che libera le notti future e quelle passate. `TestExpiredHoldConfirmation::test_conferma_dopo_che_le_notti_sono_state_rivendute`: `409`, la prima resta in attesa con la sola voce di creazione, la seconda resta confermata.

---

15 · `POST /api/v1/admin/bookings/{booking_id}/payment`

Registrazione di un incasso manuale.

Accesso	Amministrativo
Body	`AdminPaymentRegistrationSchema`
Risposta	`BookingSchema`

Codici: `200` · `401` · `403` · `404` · `409` operazione incoerente con la prenotazione · `422` contratto violato

Logica. Per i pagamenti in struttura: incasso, rimborso, correzione di una registrazione sbagliata. I pagamenti online passano dal webhook Stripe e non si toccano da qui.

Il campo `amount` non esiste più *(rimosso il 28/09/2026)*. Era accettato e scartato in silenzio: `Booking` registra quanto è **dovuto**, non quanto è stato **incassato**, e non aveva dove salvarlo (debito #21). Lo schema ora rifiuta i campi non dichiarati, quindi chi lo invia riceve `422` *«Il campo 'amount' non è previsto.»* invece di un `200` fuorviante. L'incasso registrato vale per il totale della prenotazione.

Contratto (`422`): `payment_method` solo `CASH_ON_SITE` · `POS_ON_SITE` · `BANK_TRANSFER`; `payment_status` solo `PAID` · `REFUNDED` · `PENDING`; con `PAID` il metodo è obbligatorio; **`reason` obbligatoria per `REFUNDED` e `PENDING`**

(dal 09/10/2026) — *«La motivazione è obbligatoria per un rimborso o una correzione»*, massimo 500 caratteri, spazi ai bordi tolti, solo spazi = assente.

Regole sulla prenotazione (`409` , `InvalidPaymentOperation`):

Richiesta	Consentita se	Effetto sul metodo
qualsiasi	il pagamento **non** è `STRIPE_CARD` — un pagamento online si rimborsa da Stripe e il webhook aggiorna lo stato	—
`PAID`	prenotazione `CONFIRMED` · `CHECKED_IN` · `COMPLETED` · `NO_SHOW` , e non già `PAID`	impostato
`REFUNDED`	pagamento `PAID` , **qualunque stato della prenotazione, anche** `CANCELLED` — è il caso tipico: si annulla, poi si restituisce	invariato se omesso
`PENDING`	pagamento `PAID` — correzione di un incasso registrato per errore	azzerato

Storico dei pagamenti *(dal 09/10/2026)*. Ogni cambio di `payment_status` lascia una voce in `booking_payment_history` — stato precedente e nuovo, metodo risultante, attore, motivazione — **nella stessa transazione** del cambiamento: un'operazione rifiutata o un salvataggio fallito non la lasciano. Vale per **tutti** i percorsi, non solo questo endpoint: c'è un unico punto del codice che cambia lo stato del pagamento (`BookingService._set_payment_status`), usato dall'incasso manuale, dalla creazione già pagata (`from_status = null` , *«Incasso registrato alla creazione»*) e dagli esiti di Stripe (attore `SYSTEM`). Un evento ripetuto che non cambia nulla — Stripe ripete i webhook — non lascia una seconda voce. Le voci compaiono in `BookingSchema.payment_history` , **solo nella vista amministrativa**, senza l'id dell'operatore; **nessun importo** (debito #21). Le prenotazioni pagate prima di questa data non hanno voci: lo storico non viene ricostruito a ritroso.

Concorrenza. Due operatori che registrano insieme sulla stessa prenotazione: il secondo salvataggio trova la `version` cambiata e riceve `409` (`StaleDataError`), senza effetti.

Audit: oltre alla voce di storico, ogni registrazione riuscita scrive nel log applicativo, **dopo il commit**, codice prenotazione, stato precedente e nuovo, metodo e id dell'admin — nessun dato dell'ospite e nessuna motivazione (testo libero, potrebbe contenerne).

Test: `TestOperations::test_registrazione_incasso` e l'intera classe `TestManualPayment` (8 test); classe `TestPaymentHistory` (8): voce per incasso, rimborso e correzione con motivazione, motivazione obbligatoria, nessuna voce su un'operazione rifiutata, creazione già pagata, storico assente dalle risposte pubbliche, `403` / `401` ; in `test_payment_api.py` le voci di Stripe e l'evento ripetuto; il contratto in `test_booking_schema.py::TestAdminPaymentRegistration` .

---

16 · `POST /api/v1/admin/bookings/{booking_id}/extend-hold`

Proroga del blocco temporaneo.

Accesso	Amministrativo
Body	`BookingExtendHoldSchema` — `minutes` (1–120)
Risposta	`BookingSchema`

Codici: `200` · `401` · `403` · `404` · `409` prenotazione non in attesa, o camere nel frattempo vendute

Logica. Concede più tempo a un ospite che sta completando la prenotazione. Solo su prenotazioni in stato `PENDING_*` .

Se nel frattempo le camere sono state vendute a qualcun altro la proroga viene respinta: il blocco non si può riattivare su uno slot ormai occupato, e l'exclusion constraint lo impedisce.

> **Endpoint poco usato, tenuto di proposito.** Serve a una situazione reale ma rara: l'ospite al telefono che sta compilando il modulo e sta per scadere. Non ha costi di manutenzione — nessun'altra parte del sistema dipende da lui — e il giorno in cui servisse, riscriverlo costerebbe più di quanto costi lasciarlo. Se dal frontend non verrà mai richiamato, non è un problema.

Test: `TestOperations::test_proroga_hold_su_prenotazione_confermata_rifiutata`, `test_proroga_hold_su_prenotazione_in_attesa`.

---

17 · `POST /api/v1/admin/bookings/sweep-expired`

Esegue subito lo sweeper delle scadenze.

Accesso	Admin
Body	nessuno
Risposta	`SweepResultSchema`

Codici: `200` · `401` · `403`

Logica. Esegue **una passata completa dello sweeper** (`sweep_once`), identica a quella del giro automatico e nello stesso ordine:

1. **rilascia le autorizzazioni Stripe** ancora vive sulle prenotazioni scadute; quelle che non si riesce a rilasciare restano fuori dalla passata;
2. porta a `EXPIRED` le prenotazioni temporanee con blocco scaduto, disattiva le righe camera, invalida i token, scrive la traccia storica con attore `SYSTEM`;
3. invia le email **dopo** il commit.

> ⚠️ **Fino al 02/10/2026 il punto 1 non c'era.** Questo endpoint chiamava direttamente `expire_pending`, saltando il rilascio delle autorizzazioni: poteva quindi rimettere in vendita uno slot su cui esisteva ancora un'autorizzazione viva su Stripe — esattamente ciò che la decisione #31 vieta al giro automatico.
>
> Il sistema restava corretto, perché il ricontrollo nel webhook intercettava il caso (`payment_api.md` §5.3), ma l'ospite viveva un pagamento che sembrava riuscire e poi si annullava: l'esperienza che l'incasso differito esiste per evitare. Era una svista di cablaggio — l'endpoint è dello Step F, il rilascio delle autorizzazioni è arrivato con lo Step G ed è stato aggiunto solo a `sweep_once`. Ora l'endpoint **chiama** `sweep_once`, quindi le due vie non possono più divergere.

A cosa serve. A collaudare il meccanismo senza restare quindici minuti a guardare l'orologio, e come leva operativa quando lo sweeper in background è spento (`SWEEPER_ENABLED=false`) perché lo si pilota da uno scheduler esterno.

Non è distruttivo. Libera slot che il sistema considera già liberi, e rieseguirlo non cambia nulla: la seconda passata non trova più prenotazioni in attesa scadute.

Test: `test_l_admin_puo_eseguire_lo_sweeper_a_mano`, `test_lo_sweeper_a_mano_e_idempotente`, `test_un_utente_semplice_non_puo_eseguire_lo_sweeper`, `test_senza_autenticazione_lo_sweeper_e_inaccessibile`, e in `test_payment_api.py` `test_lo_sweeper_manuale_rilascia_l_autorizzazione` — quest'ultimo è il presidio contro il ritorno del difetto, e verifica *che cosa è stato chiesto a Stripe*, non solo l'esito.

```
curl -i -X POST http://localhost:8000/api/v1/admin/bookings/sweep-expired \
  -b cookies.txt
```

```
{"expired_count": 3, "notified_count": 2, "swept_at": "2026-09-20T14:31:07Z"}
```

---

18 · `GET /api/v1/admin/bookings/planning` *(dal 09/10/2026)*

Tabellone del back-office: camere e soggiorni di un periodo.

Accesso	Admin
Query	`date_from` (incluso) · `date_to` (escluso), formato `YYYY-MM-DD`
Risposta	`PlanningSchema`

Codici: `200` · `401` · `403` · `422` date mancanti, `date_to` non successiva a `date_from`, finestra oltre `PLANNING_MAX_WINDOW_DAYS` (62)

Logica. Restituisce le camere ordinate per numero e, separatamente, i soggiorni che le **occupano** nella finestra: una voce (`PlanningStaySchema`) per ogni camera di ogni prenotazione. Il client li dispone sulle righe con `room_id` .

- **Quali soggiorni.** La regola è `_occupying_item_conditions` , la stessa di disponibilità, conflitti e calendario pubblico: compaiono le prenotazioni in attesa **con blocco valido**, confermate, con ospite arrivato e concluse (anche con partenza anticipata, perché le notti restano occupate). **Non** compaiono annullate, scadute, non presentate e attese con blocco scaduto: le loro notti sono libere. Per costruzione due soggiorni della stessa camera non si sovrappongono mai (vincolo anti-overbooking). Le prenotazioni escluse si cercano dall'elenco (endpoint 10).
- **Quali camere.** Tutte quelle in vendita, più le disattivate che hanno soggiorni nella finestra: altrimenti quelle barre non avrebbero una riga.
- **Date.** Quelle reali della riga camera, anche quando sporgono dalla finestra: il ritaglio è un problema di disegno e spetta al client. Il giorno di partenza non è occupato, quindi un soggiorno che parte il primo giorno della finestra non compare.
- **Passato.** Ammesso, a differenza del calendario pubblico: l'admin rivede anche i soggiorni conclusi.
- **Dati personali.** Solo nome e cognome e numero di ospiti: **niente email né telefono**, che restano nel dettaglio (endpoint 11). Il tabellone elenca molti ospiti insieme, e il tetto alla finestra impedisce di scaricare l'anagrafica di un anno in una richiesta.
- **Prestazioni.** Una query per le righe camera con la prenotazione in `JOIN` (`contains_eager`), una per le camere.

Ordine delle rotte. `/planning` è dichiarata **prima** di `/{booking_id}` : le rotte si confrontano in ordine, e «planning» verrebbe altrimenti letto come un identificativo, con un `422` .

Test: `tests/test_planning_api.py` (14) — `401` / `403` ; finestra invertita, vuota, oltre il limite, passato ammesso; risposta vuota e camere ordinate; una barra per camera con i dati attesi; nessuna email né telefono; soggiorno a cavallo del bordo con date reali; partenza il primo giorno esclusa; attesa valida inclusa e scaduta esclusa; annullata esclusa; non presentato escluso e conclusa inclusa; camera disattivata solo se ha soggiorni.

```
curl -b cookies.txt "http://localhost:8000/api/v1/admin/bookings/planning?date_from=2026-10-08&date_to
```

```
{
  "date_from": "2026-10-08",
  "date_to": "2026-10-22",
  "rooms": [{"id": "…", "number": 101, "name": "Camera Girasole", "enabled": true}],
  "stays": [{
    "booking_id": "…", "code": "BB-2026-000123", "room_id": "…",
    "check_in": "2026-10-12", "check_out": "2026-10-15",
    "status": "CONFIRMED", "payment_status": "PENDING",
    "guest_name": "Mario Rossi", "guest_count": 2, "hold_expires_at": null
  }]
}
```


---

5. Schemi

5.1 Richiesta

`OCCUPANCYREQUESTSCHEMA`

Campo	Tipo	Obbl.	Vincoli
`room_ids`	UUID[]	sì	non vuota, senza duplicati, max 5
`date_from`	date	sì	≥ oggi
`date_to`	date	sì	> `date_from`, finestra ≤ 92 giorni, ≤ oggi + 395 giorni

`BOOKINGQUOTEREQUESTSCHEMA`

Campo	Tipo	Obbl.	Vincoli
`check_in`	date	sì	≥ oggi, entro 365 giorni
`check_out`	date	sì	> `check_in`, da 1 a 30 notti
`guest_count`	int	sì	1–100
`room_ids`	UUID[]	sì	non vuota, senza duplicati, max 5
`payment_option`	enum	sì	`PAY_NOW` | `PAY_ON_ARRIVAL`

`GUESTDATASCHEMA`

Campo	Tipo	Obbl.	Vincoli
`firstname`	string	sì	2–50 caratteri, senza spazi esterni
`lastname`	string	sì	2–50 caratteri, senza spazi esterni
`email`	string	sì	email valida (RFC)
`phone_number`	string	sì	*(dal 10/10/2026)* qualunque Paese: 4–25 caratteri in ingresso, con spazi, trattini, punti, barre e parentesi tollerati; `00` vale come `+`; **senza prefisso si assume l'Italia (+39)**. Salvato **normalizzato in E.164** (`+393331234567` , al massimo 15 cifre dopo il `+`). **Per l'Italia solo cellulari**: `+39` , poi `3` e altre nove cifre (fissi e vecchi cellulari a 9 cifre rifiutati). Per gli altri Paesi solo la forma; la lunghezza del numero nazionale (10 cifre per tutti, per ora) la controlla il form (debito #40). Prima: esattamente 10 cifre italiane

`GUESTBOOKINGCREATESCHEMA`

Campo	Tipo	Obbl.	Vincoli
`quote_token`	string	sì	20–4096 caratteri
`guest`	`GuestDataSchema`	sì	—
`accept_terms`	bool	sì	**deve essere `true`**
`captcha_token`	string	no	max 4096
`website`	string	no	**honeypot: deve restare vuoto**

`USERBOOKINGCREATESCHEMA`

Campo	Tipo	Obbl.
`quote_token`	string	sì
`accept_terms`	bool	sì (deve essere `true`)

`BOOKINGMANAGEREQUESTSCHEMA`

Campo	Tipo	Vincoli
`token`	string	20–512 caratteri

Solo il token, senza `reason` : qui non si cancella nulla. Schema separato da `BookingCancelSchema` di proposito — i due endpoint hanno cicli di vita diversi (questo è idempotente, quello consuma il token), e un `reason` accettato su una lettura sarebbe solo un invito a fraintendere.

`BOOKINGCONFIRMSCHEMA` · `BOOKINGCANCELSCHEMA` · `OWNBOOKINGCANCELSCHEMA` · `BOOKINGLOOKUPSCHEMA`

Schema	Campi
`BookingConfirmSchema`	`token` (string, 20–512, obbligatorio)
`BookingCancelSchema`	`token` (obbligatorio) · `reason` (max 500, opzionale)
`OwnBookingCancelSchema`	`reason` (max 500, opzionale) — nessun token: basta la sessione
`BookingLookupSchema`	`code` (3–20, normalizzato in maiuscolo) · `email`

SCHEMI AMMINISTRATIVI *(ENDPOINT ALLO STEP E)*

Schema	Campi principali
`AdminBookingCreateSchema`	date, `guest_count`, `room_ids`, `payment_option`, `payment_method?`, `user_id` **XOR** `guest`, `skip_email_confirmation` (default `true`), `mark_as_paid` (default `false`), `admin_notes?` — consente date nel passato; **pagamento coerente** (vedi endpoint 12)
`BookingStatusUpdateSchema`	`new_status`, `reason?` — **obbligatoria** se `new_status = CANCELLED`
`AdminPaymentRegistrationSchema`	`payment_status` (`PAID` · `REFUNDED` · `PENDING`), `payment_method?` (solo manuali; obbligatorio con `PAID`), `reason?` (≤ 500; obbligatoria con `REFUNDED` e `PENDING`) — `extra="forbid"`: `amount` e ogni altro campo non dichiarato → `422`
`BookingExtendHoldSchema`	`minutes` (1–120)
`BookingSearchFiltersSchema`	`status[]?`, `date_from?`, `date_to?`, `email?`, `code?`, `room_id?`, `sort` (`BookingSortOrder`, default `CHECK_IN_DESC`), `page` (default 1), `page_size` (default 20, max 100)

> **Nessuno schema di input contiene un campo prezzo.** C'è un test che lo verifica.

5.2 Risposta

`AVAILABILITYRESPONSESCHEMA`

Campo	Tipo	Note
`check_in` / `check_out`	date	—
`nights`	int	notti di soggiorno
`guest_count`	int	—
`rooms`	`AvailableRoomSchema[]`	ordinate per numero camera
`suggested_combinations`	`RoomCombinationSchema[]`	max 10, minimali

`AVAILABLEROOMSCHEMA`

Campo	Tipo	Note
`id` · `name` · `number` · `capacity`	UUID, string, int, int	—
`price_per_night`	decimal string	prezzo a notte
`nights` · `subtotal`	int, decimal string	subtotale per l'intero soggiorno
`services`	`RoomServiceSchema[]`	`{id, name}`
`fits_all_guests`	bool	la camera da sola basta per gli ospiti

`OCCUPANCYRESPONSESCHEMA`

Campo	Tipo	Note
`date_from` / `date_to`	date	la finestra richiesta, `date_to` esclusa
`room_ids`	UUID[]	le camere richieste
`unavailable_nights`	date[]	notti in cui almeno una camera è occupata, ordinate; vuota se tutto libero

`ROOMCOMBINATIONSCHEMA`

`room_ids` (UUID[]) · `rooms_count` (int) · `total_capacity` (int) · `total_price` (decimal string) · `wasted_capacity` (int, posti letto eccedenti)

> Porta solo gli identificativi: i dati delle camere sono già in `rooms`.

`BOOKINGQUOTERESPONSESCHEMA`

`check_in` · `check_out` · `nights` · `guest_count` · `payment_option` · `lines[]` · `base_price` · `discount_amount` · `total_price` · `currency` · **`quote_token`** · `quote_expires_at`

`lines[]` è `PriceLineSchema`: `room_id`, `room_name`, `unit_price`, `nights`, `line_total`.

`BookingPublicSchema` — vista per l'ospite

Campo	Tipo	Note
`code`	string	codice leggibile, es. `BB-2026-A7K3QX`
`status`	enum	vedi §6
`check_in` · `check_out` · `nights`	date, date, int	—
`guest_count`	int	—
`guest_firstname` · `guest_lastname` · `guest_email`	string	snapshot anagrafico
`rooms`	`BookingRoomItemSchema[]`	—
`base_price` · `discount_amount` · `total_price` · `currency`	decimal string	—
`payment_option` · `payment_status`	enum	—
`hold_expires_at`	datetime | null	scadenza del blocco temporaneo
`cancellation_deadline`	datetime | null	termine di cancellazione gratuita
`confirmed_at`	datetime | null	—

> **Non contiene** `id`, `user_id`, `version`, `admin_notes`, campi di audit né riferimenti Stripe. C'è un test che lo verifica.

`BookingRoomItemSchema`

`room_id` · `room_name` · `room_number` · `check_in` · `check_out` · `nights` · `unit_price` · `line_total`

> `unit_price` è il prezzo **congelato** al momento della prenotazione: una modifica successiva al listino non lo altera.

`BookingManageSchema` e `GuestCancellationPolicySchema`

`BookingManageSchema` = `booking` (`BookingPublicSchema`) + `cancellation`.

`GuestCancellationPolicySchema`: `can_cancel` (bool) · `blocked_by` (enum o `null`) · `message` (string o `null`) · `free_until` (datetime o `null`).

`BookingCreatedSchema`

Campo	Tipo	Note
`booking`	`BookingPublicSchema`	—
`confirmation_token`	string | null	**solo se** `EMAIL_ENABLED=false` — vedi §3.6

`BOOKINGSCHEMA` — VISTA AMMINISTRATIVA

Tutti i campi di `BookingPublicSchema`, più: `id`, `source_channel`, `user_id`, `guest_phone`, `payment_method`, `cancelled_at`, `cancellation_reason`, `admin_notes`, `created_at`, `updated_at`, `created_by`, `last_updated_by`, **`version`**, `status_history[]`, `payment_history[]` *(dal 09/10/2026)*.

> **`version` è il contatore dell'optimistic locking**, mantenuto dal database (`version_id_col`). Oggi nessun endpoint lo accetta in ingresso: la modifica amministrativa è stata rimossa (voce 13, debito #21). Resta esposto perché sarà il valore da rimandare quando quel percorso tornerà. Volutamente assente da `BookingPublicSchema`: all'ospite non serve.

`ADMINBOOKINGCREATEDSCHEMA`

`booking` (`BookingSchema`) · `confirmation_token` (string | null).

Porta la vista completa, non quella pubblica: l'admin deve vedere audit, canale di origine e note interne. Il token è `null` quando `skip_email_confirmation` è attivo.

`BOOKINGSTATUSHISTORYSCHEMA`

`from_status?` · `to_status` · `actor_type` · `reason?` · `created_at`.

> Lo storico può registrare **anche modifiche che non cambiano stato**, con una riga in cui `from_status == to_status`. Le scriveva la `PATCH`, oggi rimossa; il meccanismo resta perché "chi ha fatto cosa e quando" è esattamente l'informazione che serve in caso di contestazione, e la modifica tornerà.

`BOOKINGPAYMENTHISTORYSCHEMA` *(DAL 09/10/2026)*

`from_status?` · `to_status` · `payment_method?` · `actor_type` · `reason?` · `created_at`.

> Timeline del **pagamento**, separata da quella della prenotazione perché descrive un'altra macchina a stati. `from_status` è `null` sulla voce di una prenotazione nata già pagata; `payment_method` è il metodo **dopo** il cambiamento (`null` dopo una correzione). Nessun `actor_id` e nessun importo.

`BOOKINGLISTITEMSCHEMA` E `PAGINATEDBOOKINGSSCHEMA`

Riga di elenco: `id`, `code`, `status`, `check_in`, `check_out`, `guest_firstname`, `guest_lastname`, `guest_email`, `guest_count`, `rooms_count`, `room_names[]`, `total_price`, `payment_status`. *(`guest_firstname`, `guest_count` e `room_names` dal 01/10/2026.)* Contenitore: `items[]`, `total`, `page`, `page_size`, `pages`.

`SWEEPRESULTSCHEMA`

Campo	Tipo	Note
`expired_count`	int	Prenotazioni portate a `EXPIRED`
`notified_count`	int	Ospiti avvisati via email
`swept_at`	datetime	Istante di esecuzione, in UTC

`notified_count` è minore o uguale a `expired_count`: le scadenze più vecchie di `SWEEPER_NOTIFY_MAX_AGE_HOURS` vengono sistemate a database ma non notificate.

`PLANNINGSCHEMA` · `PLANNINGROOMSCHEMA` · `PLANNINGSTAYSCHEMA` *(DAL 09/10/2026)*

Schema	Campi
`PlanningSchema`	`date_from` · `date_to` · `rooms` · `stays`
`PlanningRoomSchema`	`id` · `number` · `name` · `enabled`
`PlanningStaySchema`	`booking_id` · `code` · `room_id` · `check_in` · `check_out` (della riga camera) · `status` · `payment_status` · `guest_name` (nome e cognome) · `guest_count` · `hold_expires_at` (solo per le attese)

Richiesta: `PlanningRequestSchema` (`date_from` , `date_to`), validata da `validate_planning_window` .

---

6. Enumerazioni

`BookingStatus`

Valore	Significato	Slot occupato
`PENDING_CONFIRMATION`	In attesa di conferma via email	sì, finché il blocco è valido
`PENDING_PAYMENT`	In attesa del pagamento online	sì, finché il blocco è valido
`CONFIRMED`	Confermata	sì
`CHECKED_IN`	Ospite arrivato	sì
`COMPLETED`	Soggiorno concluso	sì
`CANCELLED`	Annullata *(terminale)*	no
`EXPIRED`	Blocco scaduto senza conferma *(terminale)*	no
`NO_SHOW`	Ospite non presentato *(terminale)*	**no** *(dal 07/10/2026)*

Transizioni ammesse

Da	A
`PENDING_CONFIRMATION`	`CONFIRMED` · `EXPIRED` · `CANCELLED`
`PENDING_PAYMENT`	`CONFIRMED` · `EXPIRED` · `CANCELLED`
`CONFIRMED`	`CHECKED_IN` · `CANCELLED` · `NO_SHOW`
`CHECKED_IN`	`COMPLETED`
`COMPLETED` · `CANCELLED` · `EXPIRED` · `NO_SHOW`	*(terminali)*

Ogni transizione non prevista produce `409` . Ogni transizione eseguita lascia una riga in `booking_status_history` con attore e motivazione.

`GuestCancellationBlock`

Perché l'ospite non può cancellare da sé. Compare in `blocked_by` di `POST /manage` .

Valore	Significato	Cosa mostrare
`STATUS_NOT_CANCELLABLE`	Già annullata, scaduta, conclusa, o arrivo registrato	Nessun pulsante: non c'è nulla da annullare
`NON_REFUNDABLE`	Pagata online in anticipo	Invito a contattare la struttura
`DEADLINE_PASSED`	Termine di cancellazione gratuita superato	Invito a contattare la struttura

Altri enum

Enum	Valori
`PaymentOption`	`PAY_NOW` · `PAY_ON_ARRIVAL`
`PaymentStatus`	`NOT_REQUIRED` · `PENDING` · `AUTHORIZED` · `PAID` · `FAILED` · `REFUNDED` · `PARTIALLY_REFUNDED`
`PaymentMethod`	`STRIPE_CARD` · `CASH_ON_SITE` · `POS_ON_SITE` · `BANK_TRANSFER`
`BookingChannel`	`PUBLIC_GUEST` · `PUBLIC_USER` · `ADMIN_BACKOFFICE`
`BookingTokenPurpose`	`CONFIRM_EMAIL` · `MANAGE` · `CANCEL`
`AuditActorType`	`GUEST` · `USER` · `ADMIN` · `SYSTEM`
`UserRole`	`user` · `admin`

7. Copertura dei test

`pytest` dalla root del progetto. Il database di test si crea da solo alla prima esecuzione.

File	Test	Database	Cosa verifica
`test_booking_schema.py`	96	no	Vincoli dei DTO: date, capienza, XOR utente/ospite, honeypot, normalizzazione codice, **coerenza del pagamento admin, contratto dell'incasso manuale**, **telefono internazionale (validi, non validi, normalizzazione)**
`test_pricing_service.py`	17	no	Sconti, arrotondamento `ROUND_HALF_UP`, assenza di `float`, quote token, penali
`test_availability_combinations.py`	15	no	Minimalità, ordinamento, limiti, euristica su inventari ampi; notti del calendario (giorno di partenza libero, unione, ritaglio)
`test_occupancy_api.py`	18	sì	Calendario: notti occupate, giorno di partenza libero, unione fra camere, blocco scaduto, annullate e scadute, ritaglio sulla finestra, `404` / `409` / `422` / `429`, **coerenza con `/availability`**
`test_booking_service.py`	14	sì	**Concorrenza (eseguita 10 volte)**, ciclo di vita, transizioni, hold scaduto, back-to-back
`test_booking_api.py`	34	sì	Flusso end-to-end, rate limit, protezioni, autorizzazione, **persistenza**, **email**, pagina di gestione (5 di essi sospesi con le rotte `/me`)
`test_exception_handler.py`	9	no	Errore del client (`422`) contro errore del server (`500`), cosa non finisce nella risposta, nome del campo negli errori su liste, **violazioni di vincolo (`409` / `500`) senza SQL né dati dell'ospite**
`test_admin_booking_api.py`	66	sì	Autorizzazione, creazione on-behalf-of, assenza della rotta di modifica, stato, **transizioni negate all'admin**, **regole dell'incasso manuale**, riga di audit, **ordinamento e contenuto della riga**, cronologia nelle risposte, email della conferma dall'admin, **partenza anticipata e mancata presentazione**, **storico dei pagamenti**, **conferma con notti rivendute**, **telefono salvato in E.164**
`test_room_api.py`	3	sì	Una camera con prenotazioni non si elimina, nemmeno se disattivata
`test_email_service.py`	36	no	Rendering dei template, escaping, link, mascheramento nei log, robustezza del canale, **testi coerenti con le regole**, formato dei termini
`test_planning_api.py`	14	sì	Tabellone: accesso, finestra, quali soggiorni compaiono (stessa regola della disponibilità), camere disattivate, **nessuna email né telefono**
`test_booking_expiration.py`	19	sì	Transizione a `EXPIRED`, slot riprenotabile, idempotenza, soglia di notifica, endpoint admin, scadenza e pulizia dei token
Totale eseguito	**~250**		il test di concorrenza è parametrizzato su 10 iterazioni. I test dei pagamenti sono contati in `payment_api.md`

Il test che conta più di tutti

`test_due_prenotazioni_concorrenti_una_sola_vince` lancia due creazioni identiche con `asyncio.gather` su sessioni distinte e pretende **esattamente un successo e un** `RoomNotAvailable`. È ripetuto 10 volte, perché una race condition che si manifesta una volta su dieci resta una race condition.

L'anti-overbooking non è garantito dal codice applicativo ma da un **exclusion constraint di PostgreSQL**: nessuna sequenza di operazioni concorrenti può produrre una doppia vendita.

Il test che ha trovato un bug scrivendosi

`test_il_link_ricevuto_per_email_permette_davvero_di_annullare` estrae il token dal corpo dell'email e lo usa su `POST /cancel`, esattamente come farebbe l'ospite. È l'unico test che percorre l'intera catena *emissione → email → endpoint*, ed è quello che ha reso visibile un endpoint che nessun client poteva raggiungere (§8bis.11.3).

La lezione è generale: un test che chiama il servizio direttamente dimostra che il servizio parla con sé stesso. Solo un test che si mette nei panni del chiamante dimostra che il chiamante può arrivarci.

---

8. Checklist di verifica manuale

Da eseguire su Swagger (`/docs`) a sviluppo concluso.

Prerequisiti

- [ ] Almeno due camere abilitate in `rooms`
- [ ] Un utente con `role = admin`
- [ ] `EMAIL_ENABLED=false` (per ricevere il token in risposta)

Flusso ospite — percorso felice

- [ ] `GET /availability` restituisce le camere libere con `fits_all_guests` corretto
- [ ] Le combinazioni proposte coprono gli ospiti e non contengono camere superflue
- [ ] `POST /quote` restituisce un totale coerente e un `quote_token`
- [ ] `POST /` crea la prenotazione in `PENDING_CONFIRMATION` e restituisce il token
- [ ] `POST /confirm` porta a `CONFIRMED`, azzera `hold_expires_at`, valorizza `cancellation_deadline`
- [ ] `POST /lookup` con codice ed email restituisce la prenotazione

Calendario (`GET /occupancy`)

- [ ] Camera senza prenotazioni nella finestra → `unavailable_nights: []`
- [ ] Prenotazione confermata dal 12 al 14 → notti `12` e `13`, **non** il `14`
- [ ] Due camere con prenotazioni in giorni diversi → l'unione delle notti
- [ ] Prenotazione annullata → le sue notti spariscono
- [ ] Finestra di 93 giorni → `422`; `date_from` ieri → `422`
- [ ] Camera disattivata → `409`; UUID inventato → `404`
- [ ] Senza cookie di sessione funziona lo stesso: è pubblico

Flusso ospite — casi limite

- [ ] Ripetere `POST /` sullo stesso slot → `409`
- [ ] Confermare due volte → `409` "già stata confermata"
- [ ] `POST /` con `website` valorizzato → `422`
- [ ] `POST /` con `accept_terms: false` → `422`
- [ ] `POST /quote` con `check_out` precedente a `check_in` → `422`
- [ ] `POST /quote` con più ospiti della capienza → `422`
- [ ] Ventun richieste a `/availability` in un minuto → `429` con `Retry-After`
- [ ] `POST /lookup` con email errata → `404` con messaggio generico

Autorizzazione

- [ ] `GET /me` senza cookie → `401`
- [ ] `GET /me` con cookie valido → le proprie prenotazioni
- [ ] `POST /me` come utente autenticato → la prenotazione compare in `GET /me`
- [ ] Cancellare la propria prenotazione col codice → `CANCELLED`
- [ ] Cancellare la prenotazione di un altro utente → `404`

Area amministrativa

- [ ] `GET /admin/bookings/` senza cookie → `401`; con utente normale → `403`
- [ ] Creazione per conto di terzi con ospite manuale → nasce `CONFIRMED`, nessun token
- [ ] Creazione con `user_id` → anagrafica presa dal profilo
- [ ] Creazione con `user_id` **e** `guest` insieme → `422`
- [ ] `PATCH /admin/bookings/{id}` → **`405`**: la rotta di modifica è stata rimossa (voce 13)
- [ ] Spostamento di una prenotazione: annullamento (endpoint 14) + ricreazione (endpoint 12) → il vecchio slot torna prenotabile, il nuovo risulta occupato
- [ ] Check-in registrato prima della data di arrivo → `409`
- [ ] Annullamento senza motivazione → `422`
- [ ] Annullamento → lo slot torna immediatamente prenotabile
- [ ] Conclusione prima della partenza senza motivazione → `422`; con motivazione → `200`, notti rimaste ancora occupate
- [ ] Conclusione il giorno dell'arrivo → `409`
- [ ] Non presentato → le notti tornano libere in `/occupancy` e si possono riprenotare
- [ ] Registrazione incasso in contanti → `payment_status: PAID`
- [ ] Incasso con `amount` nel body → `422` *«Il campo 'amount' non è previsto.»*
- [ ] Incasso su prenotazione annullata → `409`; rimborso sulla stessa, se era pagata → `200`, metodo invariato
- [ ] Rimborso su prenotazione mai pagata → `409`
- [ ] Rimborso o correzione senza `reason` → `422`; con `reason` → `200` e voce in `payment_history`
- [ ] Incasso → voce in `payment_history` con metodo e attore `ADMIN`, senza `actor_id`
- [ ] `POST /bookings/lookup` della stessa prenotazione → nessun `payment_history` nella risposta
- [ ] `PENDING_CONFIRMATION` → `EXPIRED` dall'admin → `409`
- [ ] `PENDING_PAYMENT` → `CONFIRMED` dall'admin → `409`
- [ ] Creazione admin `PAY_NOW` senza `mark_as_paid` → `422`

- [ ] Proroga hold su prenotazione confermata → `409`

Integrità dei dati

- [ ] Modificare il prezzo di una camera dopo una prenotazione → il totale storico resta invariato
- [ ] Cancellare una camera con prenotazioni → `409` `EntityInUse`, con «nemmeno se disattivata» nel messaggio
- [ ] Disattivare quella camera e riprovare → ancora `409`: disattivare è l'alternativa alla cancellazione, non il prerequisito
- [ ] Cancellare un utente → le sue prenotazioni sopravvivono con `user_id` a `null`

Email

- [ ] `POST /` → nei log compare l'invio di `booking_pending` con il link di conferma
- [ ] Seguire il link e confermare → arriva `booking_confirmed` con il **link di gestione**
- [ ] Usare quel link su `POST /cancel` → la prenotazione passa a `CANCELLED`
- [ ] Riusare lo stesso link → `400`: il token si spende una volta sola
- [ ] L'annullamento genera `booking_cancelled`
- [ ] Con `EMAIL_ENABLED=true`, `confirmation_token` è `null` nella risposta
- [ ] All'avvio di uvicorn compare `Sweeper avviato: intervallo ... secondi` — se manca, i log applicativi non hanno una destinazione
- [ ] Prenotare con un nome contenente `<b>test</b>` → nell'HTML compare escapato, non interpretato
- [ ] Spegnere il server SMTP e prenotare → la prenotazione nasce comunque (`201`)

Sweeper delle scadenze

- [ ] Attesa oltre 15 minuti senza conferma → stato `EXPIRED` e slot riprenotabile
- [ ] `POST /admin/bookings/sweep-expired` → `expired_count` coerente
- [ ] Rieseguirlo subito dopo → `expired_count: 0`
- [ ] Dopo lo sweep, riprenotare le stesse date sulla stessa camera → riesce
- [ ] Il link di conferma di una prenotazione scaduta non funziona più
- [ ] Nel dettaglio admin, l'ultima riga di storico ha attore `SYSTEM`
- [ ] Con `SWEEPER_ENABLED=false` nulla scade da solo, ma l'endpoint 17 funziona

Pagina di gestione

- [ ] Aprire il link `/prenotazione/gestisci?token=...` ricevuto per email e chiamare `POST /manage` → la prenotazione completa, con `can_cancel: true`
- [ ] Ripetere la stessa chiamata tre volte → sempre `200`: la lettura non consuma il token
- [ ] Annullare, poi richiamare `POST /manage` con lo stesso token → `400`
- [ ] Su una prenotazione `PAY_NOW` confermata → `can_cancel: false`, `blocked_by: "NON_REFUNDABLE"`
- [ ] Con un token di conferma al posto di quello di gestione → `400`

Token

- [ ] Confermare una prenotazione e leggere `expires_at` del token `MANAGE` a database: deve cadere alla **mezzanotte del check-in**, non della partenza
- [ ] Portare a mano un token a `expires_at = now() - 1 giorno` → dopo una pulizia forzata è **ancora in tabella** (soglia di ritenzione)

- [ ] Portarlo a `now() - 120 giorni` → dopo la pulizia **non c'è più**
- [ ] Nel log dello sweeper compare `Pulizia token: N righe scadute eliminate` al massimo una volta al giorno

---

8bis. Tre trappole da conoscere

Emerse durante lo sviluppo, e rilevanti per chi lavora su questo codice.

11.1 Un errore di validazione può essere colpa del server

L'handler globale è registrato sulla `ValidationError` di Pydantic, che viene sollevata sia quando il **client** manda dati sbagliati, sia quando è il **server** a costruire male un modello di risposta. Entrambi i casi producono oggi un `422` con un messaggio del tipo *"Il campo X è obbligatorio"*.

Se ricevi un `422` che nomina un campo che non hai inviato — e che non appartiene alla richiesta — non stai sbagliando tu: è un bug del backend nella costruzione della risposta. *(Correzione in coda al debito tecnico: gli errori di costruzione lato server dovranno diventare `500`.)*

11.2 Un `201` non dimostra che il dato sia stato salvato

SQLAlchemy 2.0 apre una transazione alla **prima query, anche di sola lettura**. La dipendenza di autenticazione legge la tabella utenti prima di entrare nell'endpoint, quindi su ogni rotta protetta la sessione risulta già "in transazione".

Il Service concludeva perciò che il commit spettasse a qualcun altro, e la scrittura non veniva mai persistita: risposta `201` con il corpo corretto, database vuoto. Corretto allo Step E chiudendo la transazione esplicitamente.

Conseguenza per i test: verificare una scrittura rileggendo il corpo della risposta non prova nulla. Serve una **richiesta successiva**, che usa una sessione diversa.

11.3 Un endpoint può essere corretto e comunque irraggiungibile

Due casi nello stesso modulo, trovati a distanza di un giorno.

`POST /me/{booking_id}/cancel` chiedeva l'`id` interno, che `BookingPublicSchema` non espone per scelta: nessuna risposta lo restituiva all'ospite, quindi nessun client poteva costruire quell'URL. Corretto allo Step E in `/me/{code}/cancel`.

`POST /cancel` cercava un token `CANCEL` o `MANAGE`, e **nessun percorso di codice ne emetteva uno**. La rotta era scritta, coperta da test di servizio e documentata; semplicemente, la credenziale che pretendeva non esisteva. Corretto allo Step F emettendo il token `MANAGE` alla conferma.

Entrambi sono sopravvissuti ai test perché quei test chiamavano il servizio direttamente, costruendosi in casa ciò che un client non può costruire. E in entrambi i casi non è stato un test a trovarli: è stato il **doverli scrivere**, cioè il primo momento in cui qualcuno si è messo nei panni di chi chiama.

Due controlli che vale la pena fare quando si aggiunge una rotta pubblica:

1. ogni parametro di percorso compare in almeno uno schema di risposta pubblico;
2. ogni credenziale richiesta viene emessa da qualche parte, e quel punto è raggiungibile dall'utente che dovrà spenderla.

---

9. Changelog

Data	Step	Modifiche
10/10/2026	—	**Telefono dell'ospite internazionale** (debito #32): `GuestDataSchema.phone_number` accetta numeri di qualunque Paese e li salva normalizzati in E.164 (`validate_international_phone`); senza prefisso si assume +39, quindi i numeri a 10 cifre restano validi. Migrazione dati `c4d8f2a61b3e` : i `guest_phone` già salvati a 10 cifre diventano `+39…` . **Per l'Italia solo cellulari** (`3` + 9 cifre). Il telefono degli utenti registrati non cambia; i numeri già salvati non vengono rivalidati. +19 test schema, +2 admin. `ruff` in `requirements-dev.txt`
10/10/2026	—	Pulizia (debito #10–#13, #16–#18): `model_config = ConfigDict(...)` al posto di `class Config` negli schemi delle anagrafiche; costante `HTTP_422_UNPROCESSABLE_CONTENT` (codice invariato); import inutilizzati rimossi; configurazione `ruff.toml` . Nessun cambiamento di comportamento: pytest senza avvisi
09/10/2026	—	**Nuovo `GET /admin/bookings/planning`** (endpoint 18): camere e soggiorni che le occupano in una finestra di al massimo 62 giorni (`planning_max_window_days`), passato compreso, senza email né telefono. Stessa regola di occupazione della disponibilità. Nuovo `BookingPlanningService` (sola lettura, composto da `build_planning_service`). +14 test
09/10/2026	—	**Difetto: la conferma dall'admin di una prenotazione con blocco scaduto e notti rivendute rispondeva `500`** — ora `409` `RoomNotAvailable` e la prenotazione resta in attesa. **Nuovo handler globale su `IntegrityError`** : `409` per sovrapposizione, duplicato e riferimento mancante, `500` per il resto, mai SQL né parametri nella risposta o nei log. Riconoscimento SQLSTATE unificato in `src/data/integrity_errors.py` . +1 test admin, +3 handler
09/10/2026	—	**Storico dei pagamenti**: nuova tabella `booking_payment_history` (migrazione `b5e8d2c47a19`) e `BookingSchema.payment_history` ; un unico punto cambia `payment_status` (`_set_payment_status`) e lascia la voce, per incassi e rimborsi manuali, creazione già pagata ed esiti di Stripe. `POST /admin/bookings/{id}/payment` : **`reason` obbligatoria per rimborso e correzione**. Composizione del `BookingService` unificata (`build_booking_service`). +8 test admin, +2 pagamenti, +9 schema
07/10/2026	—	`POST /admin/bookings/{id}/status` : **partenza anticipata** ammessa dal giorno dopo l'arrivo, con motivazione obbligatoria (nuova `StatusChangeReasonRequired` , `422`); le notti rimaste restano occupate. **`NO_SHOW` libera le camere** (fuori da `OCCUPYING_BOOKING_STATUSES`); migrazione dati `a7c3e91f5d20` per le prenotazioni già segnate. +6 test
04/10/2026	—	`POST /admin/bookings/{id}/status` : la **conferma** dall'admin emette il link di gestione e manda `booking_confirmed` (prima l'ospite confermato al telefono restava senza link), e invalida il vecchio link di conferma. Le risposte `BookingSchema` dell'area admin includono la voce di cronologia appena scritta (`_to_admin_schema` ricarica `status_history`). +6 test
03/10/2026	—	**Nuovo `GET /bookings/occupancy`** (endpoint 1b): notti non disponibili per 1–5 camere in una finestra `[date_from, date_to)` di al massimo 92 giorni (`occupancy_max_window_days`), solo date unite, pubblico con rate limit. Serve al calendario del back-office (incremento 3b). La condizione di occupazione è ora un unico metodo del repository (`_occupying_item_conditions`), condiviso da disponibilità, conflitti e calendario; la verifica delle camere prenotabili è un unico helper (`src/service/bookable_rooms.py`), condiviso da creazione e calendario. +32 test
02/10/2026	—	**Difetto: `POST /admin/bookings/sweep-expired` non rilasciava le autorizzazioni Stripe** prima di liberare gli slot, a differenza del giro automatico (decisione #31). L'endpoint esegue ora `sweep_once` , la stessa passata; la composizione dello sweeper è unica (`build_expiration_service`), condivisa con il `lifespan`

Data	Step	Modifiche
01/10/2026	—	`GET /admin/bookings/` : ordinamento `sort` (`CHECK_IN_DESC` · `CHECK_IN_ASC` · `CREATED_DESC`); la riga di elenco porta anche `guest_firstname` , `guest_count` e `room_names`
01/10/2026	—	Cancellazione di una camera con prenotazioni: il messaggio ora dice «nemmeno se disattivata» e suggerisce di disattivarla per non riceverne di nuove. Il vecchio testo veniva letto come «disattivala e poi potrai eliminarla». Nuovo `tests/test_room_api.py` (3 test)
28/09/2026	—	Riga di audit dell'incasso manuale spostata **dopo il commit** · email: «già saldato» e «importo già pagato» legati a `payment_status` , niente più «modificare» né «annulla» dove non è consentito, avviso di non rimborsabilità per `PAY_NOW` , termine scritto come «entro le 23:59 del giorno prima», «fino al» invece di «fino alle», chiusura distinta per l'annullamento della struttura · errori su elementi di lista nominano il campo (`room_ids`) invece dell'indice
28/09/2026	—	**Difetto 🔴: una prenotazione annullata dal back-office durante il pagamento veniva incassata** — ora l'autorizzazione si rilascia (`NOT_PAYABLE`) · incasso manuale: rimosso `amount` (accettato e scartato), `extra="forbid"` , solo metodi e stati manuali, regole sullo stato della prenotazione (`InvalidPaymentOperation` , `409`), rimborso consentito anche su annullata, log di audit · l'admin non può più portare a `EXPIRED` né confermare un `PENDING_PAYMENT` · creazione admin: combinazioni di pagamento incoerenti → `422` · messaggio italiano per i campi non previsti
26/09/2026	—	**`POST /bookings/manage`** : il link di gestione ora apre una pagina che mostra la prenotazione e la politica di cancellazione applicabile, senza consumare il token · `POST /cancel` verifica finalmente la **scadenza** del token · `422` e `500` separati per colpevole · `DB_ECHO` spostato in configurazione · rimosso l' `Annotated` di SQLAlchemy in `extra_service_router`
26/09/2026	—	**Rimossa la `PATCH` di modifica**: ricalcolava il totale senza sapere quanto fosse stato incassato (debito #21) · token `MANAGE` scade al **check-in** invece che alla partenza · pulizia dei token scaduti oltre i 30 giorni, agganciata allo sweeper con cadenza giornaliera · intervallo sweeper 60 → 300 s · politica di cancellazione documentata per esteso
20/09/2026	**G**	Pagamenti Stripe a incasso differito (documento a parte: `payment_api.md`) · quinto template email `booking_slot_lost` · lo sweeper rilascia le autorizzazioni prima di liberare gli slot
20/09/2026	**F**	`configure_logging()` : i log applicativi avevano un logger ma nessun handler, quindi venivano scartati in silenzio · servizio email con 4 template HTML+testo · sweeper delle scadenze in `lifespan` · `POST /admin/bookings/sweep-expired` · **token `MANAGE` finalmente emesso: `POST /cancel` era irraggiungibile** · invio post-commit con `BackgroundTasks` · `FOR UPDATE SKIP LOCKED` sullo sweeper · nessuna email su `PENDING_PAYMENT`
20/09/2026	**E**	7 endpoint amministrativi · **`POST /me/{code}/cancel` ora usa il codice invece dell' `id` interno** (era inutilizzabile dal client) · modifica di date e camere con ricalcolo prezzo · optimistic locking esposto al client (`version`) · `ConcurrentModification` · storico esteso alle modifiche non di stato
20/09/2026	**D**	9 endpoint pubblici e utente · rate limiting · captcha Turnstile · honeypot · `Retry-After` · `BookingCreatedSchema` · `trusted_proxy_count`
20/09/2026	**C**	Preventivo firmato · token monouso · motore di prenotazione con locking a 3 livelli · disponibilità con combinazioni
19/09/2026	**B**	21 DTO · 3 repository · `AppException` con handler unico · correzione 500→422 sulle validazioni · `EntityInUse`
19/09/2026	**A**	Modelli e macchina a stati · exclusion constraint anti-overbooking · foreign key composita · guest booking

10. Da completare

Step	Contenuto	Impatto su questo documento
G	Pagamenti Stripe	Endpoint webhook e creazione Payment Intent; codici `402`; l'email di conferma per le prenotazioni `PAY_NOW` partirà da lì
#21	Modifica della prenotazione	Colonna `amount_paid`, calcolo del conguaglio, email di modifica. Fino ad allora la rotta resta rimossa
#22	Carta a garanzia e penale	`SetupIntent` con mandato, addebito off-session su disdetta tardiva e no-show
—	Outbox pattern	Oggi un'email persa fra commit e invio non lascia traccia. Con un outbox l'invio diventa ritentabile

Documento generato automaticamente dal sorgente `docs/api/booking_api.md`. Per segnalare una discrepanza fra documento e codice, fa fede il codice.